Survey on Efficient Large Language Models: Principles, Algorithms, Applications, and Open Issues

Jian Cheng, Haidong Kang, *Student Member, IEEE*, Yuxin Shao, Nan Li, Pengjun Chen, Rui Wang, Saiqin Long, *Member, IEEE*, Xiaochun Yang, *Senior Member, IEEE*, and Lianbo Ma, *Senior Member, IEEE*

***Abstract*—With the rapid advancement of large language models (LLMs) in both academia and industry, their growing size and complexity have introduced significant challenges in terms of computational cost and deployment efficiency. To address these issues, a wide range of inference optimization techniques—including but not limited to model compression—have been proposed to accelerate LLM inference while preserving model performance. This survey provides a comprehensive overview of LLM inference acceleration strategies, analyzing them from multiple perspectives, including foundational principles, algorithmic techniques, real-world applications, and open research challenges. We begin by introducing core concepts underlying inference optimization and propose a new taxonomy that categorizes existing approaches, including quantization, pruning, distillation, efficient architectures, compilation, and hardware-aware methods. Following the lifecycle of LLM development and deployment, we examine how these techniques interact with model training, fine-tuning, and serving. Furthermore, we highlight key applications of efficient LLMs and discuss emerging trends and unresolved issues in the field. By synthesizing recent advances, this survey aims to provide actionable insights and practical guidance for researchers and practitioners working with scalable and efficient LLM systems.**

***Index Terms*—Deep learning, foundation model, large language models (LLMs), model acceleration, model compression, vision transformer (ViT).**

Received 6 February 2025; revised 23 July 2025; accepted 28 October 2025. Date of publication 14 November 2025; date of current version 4 May 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFC2907600, in part by the National Natural Science Foundation of China under Grant 62472079 and Grant 52574193, and in part by the Science and Technology Innovation and Entrepreneurship Project of TDTEC under Grant 2025-TD-ZY001 and Grant 2024-TD-ZD016-04. *(Jian Cheng and Haidong Kang contributed equally to this work.) (Corresponding author: Lianbo Ma.)*

Jian Cheng is with the Research Institute of Mine Artificial Intelligence, Chinese Institute of Coal Science, Beijing 100013, China, also with the State Key Laboratory of Intelligent Coal Mining and Strata Control, Beijing 100013, China, and also with Tiandi Science and Technology Company Ltd., Beijing 100013, China (e-mail: jiancheng@tsinghua.org.cn).

Haidong Kang, Yuxin Shao, Nan Li, Pengjun Chen, Xiaochun Yang, and Lianbo Ma are with the College of Software, Northeastern University, Shenyang, Liaoning 110819, China (e-mail: hdkang@stumail.neu.edu.cn; 2271407@stu.neu.edu.cn; 2010500@stu.neu.edu.cn; pjchen@stumail.neu.edu.cn; yangxc@mail.neu.edu.cn; malb@swc.neu.edu.cn).

Rui Wang is with the National University of Defense Technology, Changsha, Hunan 410073, China (e-mail: ruiwangnudt@gmail.com).

Saiqin Long is with the College of Information Science and Technology, Jinan University, Guangzhou 510632, China (e-mail: saiqinlong@jnu.edu.cn).

Digital Object Identifier 10.1109/TNNLS.2025.3628671

I. Introduction

LARGE language models (LLMs), exemplified by ChatGPT [1] and Claude [2], have demonstrated promising performance and great potential in natural language processing (NLP), including translation [3], summarization [4], question answering [5], and conversational agent [6]. However, due to the significant increase in terms of size and complexity, the development and deployment of LLMs require massive computational and memory resources, which leads to a series of challenges for their wide use in resource-constrained scenarios.

1) Resource Intensity: The computational power required for training and running LLM models can be prohibitively high. For example, the GPT-175B [7] model with 175 billion parameters requires a minimum of 350 GB of GPU memory for execution in half-precision (FP16), and needs five NVIDIA GeForce A100 80G GPUs just for inference. This involves demands for substantial processing power, memory, and storage, which can be a barrier to widespread applications, particularly in resource-constrained environments.
2) The inference speed of LLMs is critical for supporting real-time applications, such as interactive chatbots (e.g., ChatGPT) and live translation services (e.g., Microsoft Translator), which demand strict latency constraints. However, as shown in Table I (a curated challenging text classification task on a single NVIDIA Jetson TX2), when the model's memory consumption exceeds certain hardware limits, inference speed can degrade sharply. For instance, on the TX2 device, inference latency increases by 189–224× while the model size only grows by 5.8–12.2×. This significant latency increase negatively affects user experience and the overall effectiveness of such applications, highlighting the urgent need for optimization techniques that accelerate LLM inference without sacrificing performance.
3) Energy Consumption: The environmental impact of training and operating these models cannot be ignored. The energy consumption associated with large-scale computations contributes to the overall carbon footprint, highlighting the need for more energy-efficient

2162-237X © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

TABLE I

PARAMETER SIZE, ACCURACY, PEAK MEMORY USAGE, AND INFERENCE LATENCY OF LLM VARIANTS WITH TX2 MOBILE DEVICES

Model name	# of Params (B)	Accuracy	PeakMem. (GB)	Infer. Time (ms)
GPT2	0.14	49.8	0.5	37.64 (1x)
GPT2-Large	0.8	53.7	3.1	8065 (214x)
mT5-Small	0.3	76.4	0.7	31 (1x)
mT5-Base	0.58	83.8	1.5	2134 (69x)
mT5-Large	1.2	87.0	3.9	7214 (230x)
T5-Small	0.06	79.1	0.2	27 (1x)
T5-Base	0.22	85.4	0.8	37 (1.4x)
T5-Large	0.73	86.7	2.8	7098 (263x)

TABLE II

CARBON EMISSIONS OF LLMS

Model name	Number of parameters	Carbon intensity of grid used	Power consumption	CO_2eq emissions
GPT-3	175B	429 gCO_2eq/kWh	1,287 MWh	502 tonnes
Gopher	280B	330 gCO_2eq/kWh	1,066 MWh	352 tonnes
OPT	175B	231gCO_2eq/kWh	324 MWh	70 tonnes
BLOOM	176B	57 gCO_2eq/kWh	433 MWh	25 tonnes

approaches. Table II shows the carbon footprints of LLMs (based on [8]), including well-known models such as OPT-175B, GPT-3, Gopher, and BLOOM.

The growing demand for real-time, large-scale applications of LLMs in practical applications such as healthcare, finance, entertainment, and customer service underscores the need for techniques to accelerate inference [9], [10], [11], [12], [13]. Lightweight LLM not only improves user experience by reducing latency but also lowers operational costs and resource consumption during the training process. Efficient inference mechanisms are essential for deploying LLMs in resource-constrained environments like mobile devices and edge computing platforms. Moreover, it can boost the development of green AI. Existing reviews of LLMs [14], [15], [16], [17], [18] either target the general development of LLMs or focus on the method comparisons from the perspective of model compression, frameworks, data, and mobile edge intelligence. This highlights the need to revisit the efficient LLMs from the broad range of topics of basic principles, comprehensive algorithms, classic applications, and key open issues. To facilitate the analysis of LLMs for lightweight applications via inference acceleration, in this article, we propose an LLM inference acceleration pipeline as shown in Fig. 1. In particular, we present four main applications that have the most use of LLMs as the focus of our survey (i.e., language tasks). Overall, we divide the methods of LLMs inference acceleration lightweight applications into two types: Model-oriented inference acceleration mechanisms for LLMs, and nonmodel-oriented inference acceleration mechanisms for LLMs. For model-oriented inference acceleration mechanisms for LLMs, these mechanisms include LLMs selection, which explores LLMs that are suitable for a lightweight application, and LLMs optimization, which uses several technologies, i.e., pruning, distillation, decomposition, and quantization, to reduce the complexity of LLMs. In addition, nonmodel-oriented inference acceleration mechanisms for LLMs (i.e., early exit, KV-cache optimization, and special software for LLMs) are another series of effective methods, which do not change the architecture of LLMs. Next, we will discuss our findings and insights based on this pipeline in each section one by one. By providing a thorough analysis, this article

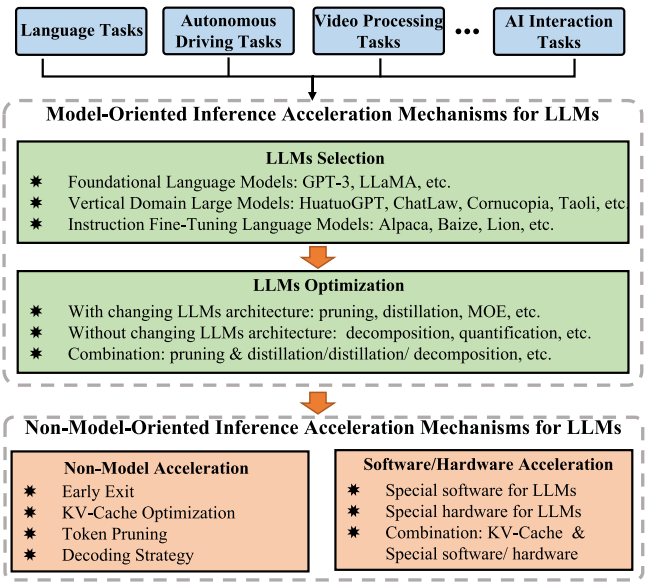


Fig. 1. Pipeline of LLM inference acceleration.

aims to provide in-depth insights into the optimization of LLM inference and promote the development of more efficient and practical AI systems. Fig. 2 illustrates the organization of this survey.

In summary, the key contributions of this article are as follows.

1) *Comprehensive Overview:* We thoroughly summarize inference acceleration for LLMs, clarifying how various methods can accelerate LLMs for real-time applications.
2) *Future Directions:* We envisage crucial opportunities for further progress in the LLM field, encouraging the exploration of model interpretability and privacy protection as a new state-of-the-art in LLM inference acceleration.

The subsequent sections of this article are organized as follows. Section II provides an overview of LLMs, detailing their introduction, basic architecture, and construction process. Section III discusses model compression principles, including techniques such as pruning, distillation, quantization, and low-rank decomposition. Section IV explores high-efficiency model architectures, focusing on efficient attention mechanisms, attention-free models, MoE models, and hardware-optimized attention. Section V covers various inference acceleration algorithms and frameworks, including early exit strategies, KV-cache optimization, decoding strategies, and token pruning. Section VI highlights the application of lightweight LLMs in real-world scenarios.

A. LLM Construction Process

Modern LLMs are generally built upon the transformer architecture using auto-regressive decoding. While the foundational architecture remains consistent, different models such as GPT and large language model meta AI (LLaMA) introduce structural and functional enhancements. In this section, we first introduce two core technical foundations (i.e., transformer and LLaMA) as key modules in the LLM construction process.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

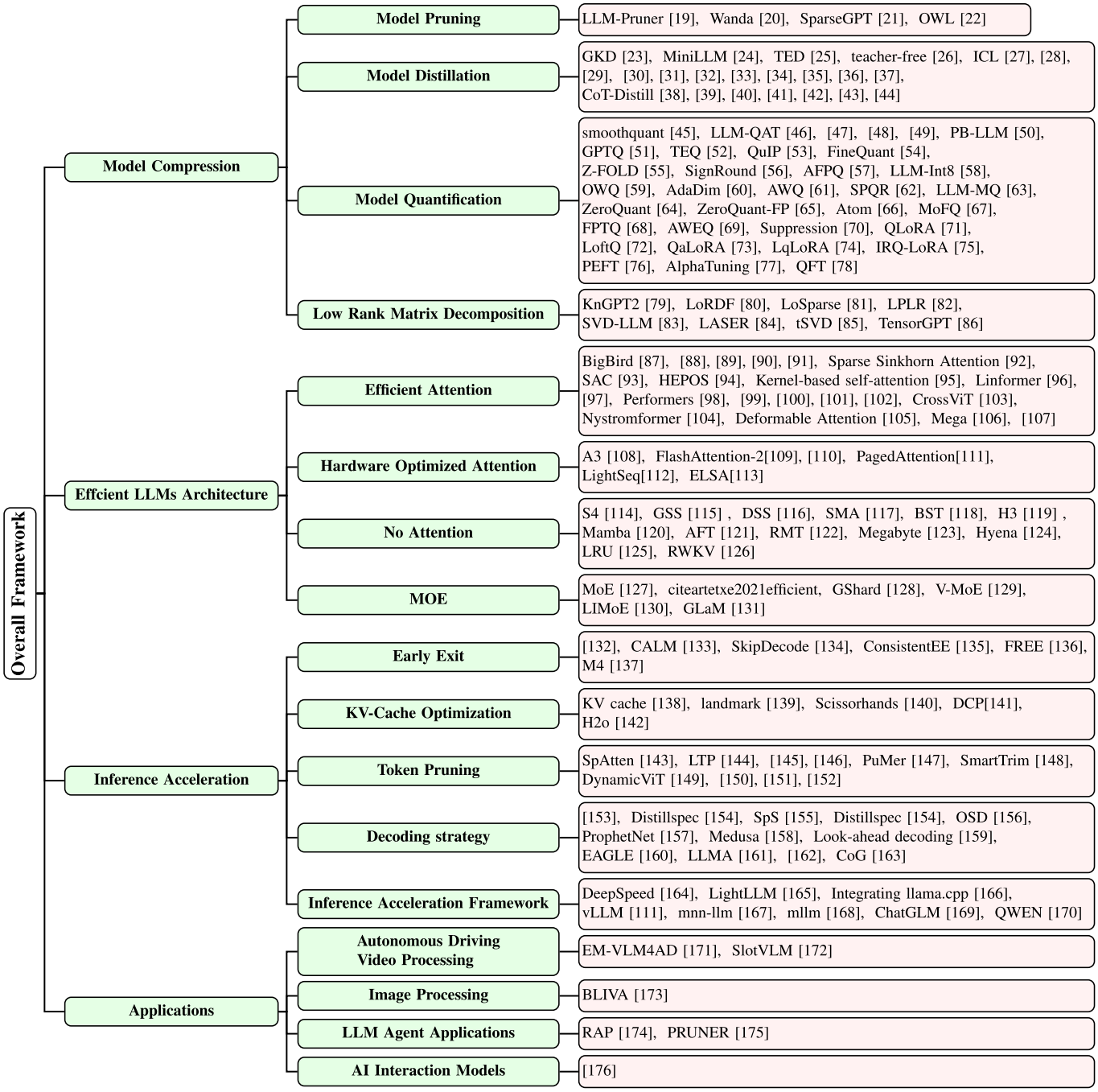


Fig. 2. Overview of this survey.

Then, we briefly outline the standard four-phase development pipeline: pretraining, supervised fine-tuning, reward modeling, and reinforcement learning.

1) Transformer Model: The transformer model [177], proposed by Google in 2017, serves as the backbone of nearly all modern LLMs. The key modules include as follows.

Embedding Module: Each word in the input sequence is first mapped to a vector representation. To compensate for the lack of sequential bias in the transformer, positional encodings are added to the embeddings to preserve order information.

Self-Attention Module: The core mechanism for context modeling. Given query (Q), key (K), and value (V) matrices, attention is computed as

$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \tag{1}$$

This allows the model to attend to different parts of the sequence. Multihead attention extends this mechanism by enabling multiple attention to be computed in parallel.

Feed Forward Network (FFN): Each token is further transformed by a two-layer fully connected module with nonlinear activation

$$y_{\text{FFN}} = \text{FFN}\left(\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V\right). \tag{2}$$

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

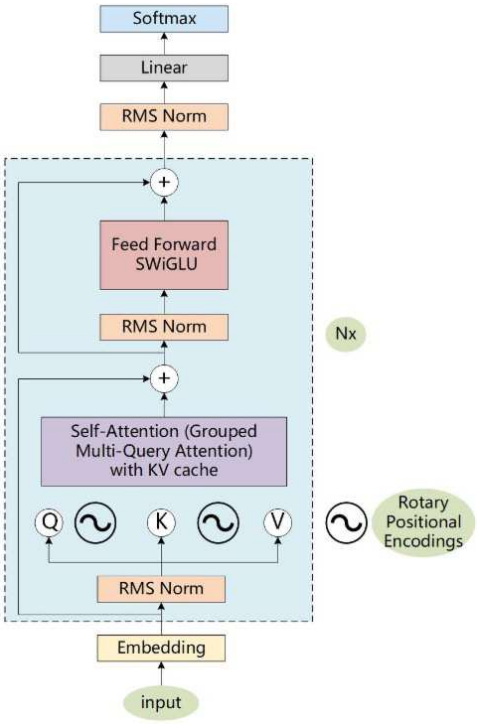


Fig. 3. Framework of LLaMA.

Residual Connections and Layer Normalization: To improve training stability, residual connections and layer normalization are applied around both the attention and FFN, addressing gradient vanishing and convergence issues.

2) LLaMA Model: LLaMA [178] introduces several key architectural refinements over the standard transformer to improve training stability and efficiency, as summarized in Fig. 3. These include: 1) prenorm residual structure; 2) RMSNorm normalization [179]; 3) swish gated linear unit (SwiGLU) activation [180]; and 4) rotary positional embeddings (RoPEs) [181]. Collectively, these components enhance the model's capability for stable optimization and efficient scaling, which are critical for LLMs.

Unlike GPT-style postnorm structures, LLaMA adopts a prenorm design, placing normalization before the self-attention and feedforward modules. Residual connections follow each computation block, facilitating better gradient flow in deep networks. The use of RMSNorm, a simplified variant of LayerNorm, reduces computational overhead while maintaining effective normalization.

In the FFN, LLaMA replaces rectified linear unit (ReLU) with the SwiGLU activation function, originally proposed in PaLM [182]. SwiGLU improves the model's nonlinear expressiveness, which is essential for capturing complex patterns in language modeling. The FFN computes as follows, where x is the token-wise hidden state and W, V, and W_2 are trainable matrices:

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x, W, V, W_2) = (\mathrm{Swish}(xW) \otimes xV) W_2. \quad (3)$$

To better show the superior performance of SwiGLU, we provide more activation functions [i.e., gated linear unit (GLU), Bilinear, rectified GLU (ReGLU), Gaussian error GLU (GEGLU), and Gaussian error linear unit (GELU)] for comparison. The definitions of those functions are as follows:

$$\begin{aligned}
\mathrm{FFN}_{\mathrm{GLU}}(x, W, V, W_2) &= (\sigma(xW) \otimes xV) W_2 \\
\mathrm{FFN}_{\mathrm{Bilinear}}(x, W, V, W_2) &= (xW \otimes xV) W_2 \\
\mathrm{FFN}_{\mathrm{ReGLU}}(x, W, V, W_2) &= (\max(0, xW) \otimes xV) W_2 \\
\mathrm{FFN}_{\mathrm{GEGLU}}(x, W, V, W_2) &= (\mathrm{GELU}(xW) \otimes xV) W_2. \quad (4)
\end{aligned}$$

By comparing the difference between these functions, we can find that the superiority of LLMs is attributed to the Swish function, which can enable LLMs to achieve a more powerful nonlinear ability than other competitors.

3) LLM Training Pipeline Overview: After architecture design, LLMs typically undergo a four-stage training and alignment process.

a) Pretraining: The model learns general language patterns from massive unlabeled text using an autoregressive objective. This stage requires significant computational resources and typically takes weeks on large-scale GPU clusters.

b) Supervised Fine-Tuning: After pretraining, the model captures general language patterns but still requires task-specific adaptation. Supervised fine-tuning addresses this by using small yet high-quality datasets, typically ranging from thousands to hundreds of thousands of carefully annotated examples. "Small" refers to the size being significantly smaller than pretraining corpora, while "high-quality" denotes data that is task-relevant, diverse, and labeled by humans or trusted sources. These datasets contain prompt-response pairs tailored for tasks such as question answering or text classification. By aligning input–output formats to specific task requirements, the model efficiently adapts to new tasks with minimal data and computation.

c) Reward Modeling: A separate model is trained to rank multiple candidate responses based on human preferences. This model later guides the reinforcement learning stage.

d) Reinforcement Learning (e.g., RLHF): The model is further fine-tuned to maximize the reward score, improving alignment with human intentions and values. This phase, while less compute-intensive than pretraining, plays a crucial role in instruction-following capabilities.

This four-stage pipeline, exemplified by models such as ChatGPT and Claude, highlights the computational and algorithmic complexity of building LLMs. In the following paragraph, we analyze the cost associated with each phase and discuss the importance of optimizing efficiency for practical deployment.

Cost Analysis of LLMs: The success of those LLMs is rooted in the scaling law: their performance consistently improves with more parameters. However, this leads to another issue, namely, a huge computational burden. How to reduce the costs of LLMs is becoming a serious challenge. To alleviate this issue, we analyze the computational and storage costs with the main components (i.e., attention, FFN, and embedding LM head) of LLMs as shown in Fig. 4. In this section, we use BERT-base, T5-small, Llama-1, and GPT-2 as main foundation models. Regarding weight consumption: we can draw the following observations: 1) the component of "FFN" occupies over 50% proportion of storage costs; 2) the component of

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

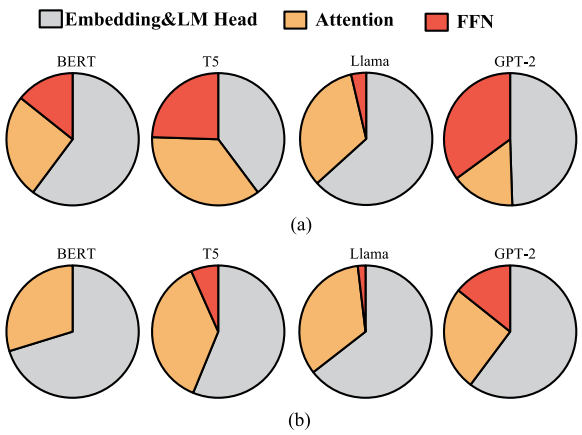


Fig. 4. Weight consumption and FLOPs cost of different LLMs. (a) Weight proportion. (b) FLOPs proportion.

"Attention" occupies over 20% proportion of storage costs, due to its composition of two fully connected layers; and 3) the component of "Embedding LM Head" is dominant in "BERT", "GPT-2", and "T5". In the view of FLOPs proportion, we can draw the same observations as weight consumption, except for "BERT". Based on the above observations, we can draw the conclusion that the huge computational and storage costs can be reduced by compressing the primary components of LLMs. In Section II, according to the characteristics of LLMs, we first provide key compression principles to guide how to design those methods, and then give a detailed discussion along taxonomy and a survey of existing approaches.

II. LLM Compression

This section provides a detailed analysis of the key principles behind compression techniques (i.e., pruning, quantization, knowledge distillation (KD), and low-rank decomposition), focusing on the characteristics of mainstream methods used in LLM compression. First, we provide a unified comparison of those compression techniques (as depicted in Fig. 5), and comprehensively discuss it from "compression principles." Then, based on the compression principles, we conduct a taxonomy and survey of existing compression-based approaches, as well as provide a comprehensive discussion of each compression technique.

A. Compression Principles

To tailor LLMs for specific applications, certain core principles of model compression should be obeyed. This chapter outlines these foundational principles by summarizing various compression methods [i.e., pruning, quantization, distillation, and low-rank matrix decomposition (LRMD)], offering a comprehensive guide for optimizing LLMs for efficient inference and deployment. With these principles, users can more easily compress and deploy LLMs in targeted scenarios, ensuring optimal performance under resource constraints.

1) Pruning Principles: Pruning neural networks involves eliminating less critical connections to reduce model complexity, improve inference efficiency, and lower computational costs. Pruning should obey the principles as follows.

1) The pruning process should minimize accuracy degradation, and performance can be recovered through evaluation metrics and fine-tuning, e.g., [19].
2) Robust and interpretable metrics should be employed to measure the importance of model components, such as layers, and attention heads, i.e., [19], [20], and [21].
3) Pruning strategies should balance the reduction of redundant parameters with the retention of sufficient model expressiveness to ensure task performance, i.e., [20] and [22].
4) In addition, pruning methods must align with hardware constraints and capabilities to maximize computational efficiency and facilitate deployment in the future study.

2) Distillation Principles: KD in LLMs not only transfers their fundamental capabilities but also contains emergent abilities unique to LLMs. KD tailored for LLMs should obey the principles as follows.

1) The student model must effectively learn the teacher model's knowledge, ensuring consistency in predictions or hidden layer features, i.e., [23] and [24].
2) The structural and capacity differences between the teacher and student models should be moderate, i.e., [25].
3) In multiteacher distillation, teacher models should provide diverse knowledge to avoid redundancy or conflicts, ensuring clear distillation objectives.
4) Both teacher and student models must avoid learning biased, harmful, or unreliable information, i.e., [32].
5) While reducing the complexity of the student model, efforts should be made to preserve its performance, ensuring usability in real-world applications, i.e., [31].

3) Quantization Principles: Quantization in LLMs primarily involves reducing the precision of weights and activations. This process offers significant benefits in terms of model efficiency and computational performance. Quantization tailored for LLMs should obey the principles as follows.

1) Quantization methods should aim to reduce model complexity (e.g., memory usage and inference latency) while minimizing accuracy degradation, i.e., [45], [46], and [51].
2) The quantization workflow should be automated to handle large-scale models efficiently, reducing the need for extensive manual tuning, i.e., [59] and [61].
3) Quantization strategies must consider LLM-specific features such as large parameter counts, deep layer structures, and sensitivity to precision changes in certain components (e.g., embedding layers), i.e., [61] and [69].
4) Quantization methods should avoid incurring excessive computational costs, such as high memory requirements or prolonged training times, i.e., [64] and [65].

4) LRMD Principles: LRMD is promising methods to reduce the computational costs of LLMs, it should obey the principles as follows.

1) LRMD should reduce model size and computational cost while minimizing the impact on model accuracy, i.e., [79].

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

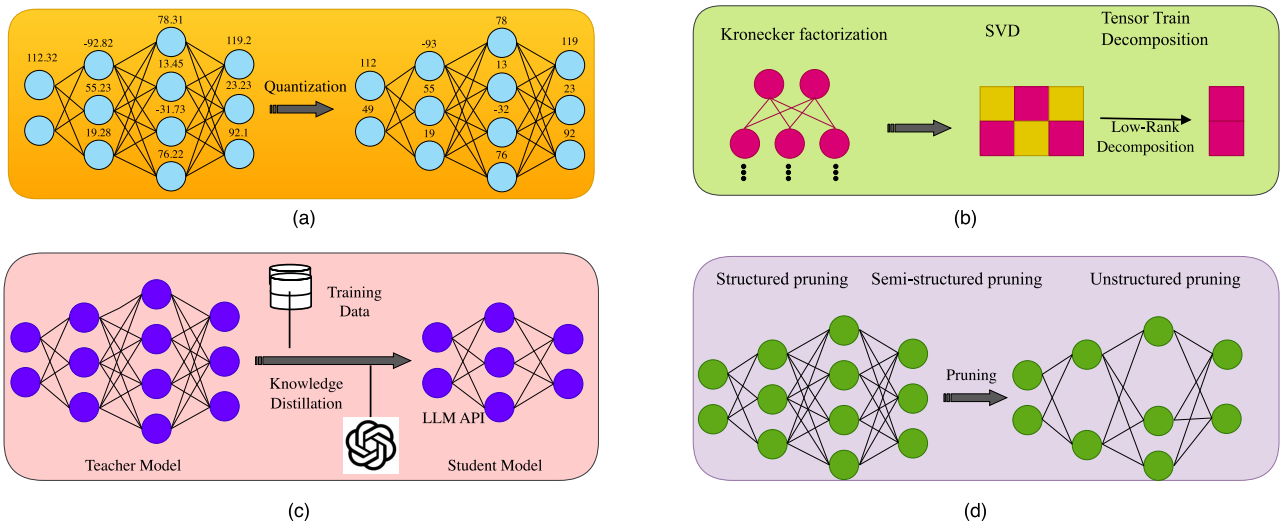


Fig. 5. Comparison of compression methods for LLMs. (a) Quantization. (b) Low-rank decomposition. (c) KD. (d) Pruning.

2) The design of LRMD must avoid excessive computational costs, including memory usage or training time (e.g., GPU days), to ensure practical scalability, i.e., [79], [81], and [84].
3) The design of LRMD should maintain robust performance across various tasks and domains, ensuring the decomposed model's transferability, i.e., [85] and [86].

B. Model Pruning

1) Problem Statement: Model pruning aims to remove redundant layers, connections, channels, or individual weights from a pretrained LLM while maintaining overall performance. Given a model with L layers, denoted as $L = \{l_0, l_1, \ldots, l_{L-1}\}$, the global pruning sparsity target is $P \in [0, 1]$. The layer-wise pruning ratio p_j can be allocated proportionally according to layer saliency scores s^j_{prune}

$$p_j = P \times L \times \frac{s^j_{\text{prune}}}{\sum_{i=0}^{L-1} s^i_{\text{prune}}}. \tag{5}$$

The core of pruning lies in evaluating *importance scores* or *saliency metrics* Score(·) to rank weights, neurons, or blocks for removal.

2) Unified Importance Scoring Framework: Most pruning methods define a score function for parameters or parameter groups based on multiple criteria

$$\begin{aligned}\text{Score}(w) = {} & \alpha \cdot f_{\text{weight}}(w) + \beta \cdot f_{\text{activation}}(a) \\ & + \gamma \cdot f_{\text{gradient}}(g) + \delta \cdot f_{\text{second-order}}(H) + \cdots\end{aligned} \tag{6}$$

where $f_{\text{weight}}(w)$ is usually weight magnitude, e.g., $|w|$ or group norm $\|W_g\|_2$, $f_{\text{activation}}(a)$ is an activation-based metric, e.g., mean absolute activation $\mathbb{E}[|a|]$, $f_{\text{gradient}}(g)$ utilizes gradient magnitude or sensitivity, $f_{\text{second-order}}(H)$ leverages Hessian-based approximations of loss change, and α, β, γ, and δ are weighting coefficients.

3) Taxonomy of Pruning Methods:

a) Structured pruning: Removes entire structures such as attention heads, neurons, or layers. A common scoring metric is the L_2-norm of weight groups

$$\text{Score}_{\text{group}} = \|W_g\|_2 \tag{7}$$

where W_g represents a group of parameters. Groups with lower scores are pruned. For example, LLM-Pruner [19] uses gradient-based one-shot pruning, and LoRA [183] fine-tunes with low-rank adaptation to recover accuracy after pruning.

b) Semi-structured pruning: Enforces fixed sparsity patterns such as $N : M$ pruning, removing the smallest N weights from every group of M

$$\text{Score}_k = |w_k|, \quad \text{prune lowest } N \text{ in each } M\text{-sized group.} \tag{8}$$

E-Sparse [184] and RIA [185] improve this by incorporating activation entropy and combined weight-activation scores

$$\text{Score}_k = \alpha|w_k| + \beta\mathbb{E}[|a_k|]. \tag{9}$$

c) Unstructured pruning: Removes individual weights based solely on importance scores, typically weight magnitude

$$\text{Score}_k = |w_k|. \tag{10}$$

SparseGPT [21] refines this by approximating the loss increase via the Hessian diagonal

$$\Delta\mathcal{L} \approx \frac{1}{2} w^T H w \tag{11}$$

where H is the Hessian matrix. Computing H exactly is costly; Wanda [20] uses heuristics leveraging outliers and activations. OWL [22] assigns nonuniform pruning ratios per layer according to outlier scores

$$p_j = \frac{\text{OutlierScore}(l_j)}{\sum_{k=1}^{L} \text{OutlierScore}(l_k)} \cdot P \tag{12}$$

ensuring critical layers are preserved while redundant ones are aggressively pruned.

4) Pruning Decision: Given the importance scores, pruning decisions are typically made by thresholding or ranking

$$z_k = \mathbf{1}[\text{Score}(w_k) > \tau] \tag{13}$$

where τ is a pruning threshold set to meet global or layer-wise sparsity constraints, and z_k is a binary mask indicating whether to keep (1) or prune (0) the parameter.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

a) Discussion: Methods such as LLM-Pruner remove entire blocks to reduce complexity, while LoRA applies parameter-efficient fine-tuning without removing parameters. Despite advances, challenges remain in balancing pruning aggressiveness and performance retention. Future work should explore adaptive sparsity schedules, hardware-aware pruning optimizations, and integrated frameworks unifying structured and unstructured sparsity under principled optimization.

C. Model Distillation

1) Problem Statement: KD aims to transfer knowledge from a larger LLM F_l to a lightweight LLM F_s while maintaining performance. To illustrate KD clearly, we provide a unified definition as follows:

$$L_{\mathrm{KD}} = D(F_l, F_s) \tag{14}$$

where $D(\cdot)$ denotes the KD methods [186], [187], [188], [189].

2) Taxonomy: Existing KD methods for LLMs are categorized into general capability distillation, and emergent capability distillation. The details are as follows.

General Capability Distillation: It aims to align student models with larger teacher models while addressing distribution mismatches and memory limitations. General knowledge distillation (GKD) [23] manages distribution mismatches by generating student samples within the teacher's range, MiniLLM [24] minimizes reverse Kullback-Leibler divergence (KLD) with policy gradients to improve text generation, and TED [25] distills task-specific knowledge from intermediate layers. In addition, a teacher-free method [26] reduces memory usage by pretraining a truncated model version.

Emergent Capability Distillation: It concentrates on endowing the student model with abilities present in large models but absent in smaller counterparts. In-context learning (ICL) [27], [28] capability of LLM refers to their outstanding adaptability in downstream NLP tasks, achieved by providing specific task descriptions or paired context information, without requiring additional training or gradient updates. However, ICL suffers from the issue of reasoning. In contrast, the chain-of-thought (CoT) [29], [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], [42], [43], [44] distillation allows models to learn from examples without the need to retrain for new tasks. In brief, the CoT prompting strategy enhances ICL by introducing intermediate reasoning steps.

Discussion: General capability distillation aims to align the outputs of a smaller student model with those of a larger teacher model. This approach addresses distribution mismatches and memory limitations. First, the design of existing methods obeys the distillation principles. GKD and MiniLLM enhance performance through improved alignment. In addition, TED incorporates knowledge from intermediate layers for better task-specific adaptability. However, this method can be resource-intensive and may struggle with complex tasks if the student model lacks capacity. In contrast, emergent capability distillation equips smaller models with advanced abilities, such as ICL and CoT reasoning. This allows smaller models to perform complex tasks without retraining. However, challenges include potential limitations in scalability and generalization. While these methods enhance model performance, balancing their strengths and weaknesses is crucial for LLM scaling.

D. Model Quantization

LLMs are usually trained and inference with high precision (i.e., 32 floating-point precision), thus restricting their practical applications. For instance, deploying a GPT-3 175B model stored in FP16 format necessitates at least $5 \times$ 80GB A100 GPUs and 350 GB of memory [45]. To address this limitation, quantization-based methods strive to significantly compress models while tolerating acceptable accuracy loss. Quantization aims to convert weights and activations into low-bit integers, thereby minimizing memory usage and computational costs by substituting high-precision floating-point parameters with lower precision integer parameters.

1) Problem Statement: To enhance the clarity of quantization for LLM, we provide a unified definition. Given an LLM with L layers, it can be formulated as $L = \{l_0, l_1, l_2, \ldots, l_{L-1}\}$. Then, we assume $B = \{B1, B2, \ldots, B32\}$ as the search space of bit-width for LLM quantization, and quantization sensitivity (i.e., the MSE between the output of the original layer and the output of the quantized layer) is defined as s^i_{quant}. The optimized quantization bit-width of l_j layer can be defined as

$$b_j = B_j + g\left(s^j_{\mathrm{quant}} \geq \frac{\sum_{i=0}^{L-1} s^i_{\mathrm{quant}}}{L}\right) \tag{15}$$

where $g(\cdot)$ denotes the indicator function.

2) Taxonomy: We categorize these approaches from several aspects, i.e., quantization-aware training (QAT), posttraining quantization (PTQ), and others.

a) Quantization-aware training: It plays a crucial role in reducing model precision while maintaining accuracy, but it faces challenges such as high computational costs and limited access to original data. To address these, LLM-QAT [46] uses data-free distillation by leveraging a full-precision model to generate data for a quantized student model. This approach achieves promising results on LLaMA with 4-bit quantization. Similarly, BitDistiller [47] combines QAT with KD to enhance sub-4-bit LLMs and reduce training costs. For more extreme quantization, methods like BitNet [48] and OneBit [49] employ 1-bit quantization while preserving model precision through binary weight techniques. PB-LLM [50] further refines this by selectively preserving high-importance weights, achieving a tradeoff between complexity and performance.

b) Posttraining quantization: PTQ offers an efficient method for compressing LLMs by reducing precision without retraining. Unlike QAT, PTQ requires no annotated data and lower computational costs, making it highly practical. In LLMs, PTQ typically targets weights (model parameters) and activations (intermediate outputs). Depending on the approach, PTQ can focus on weight-only quantization or combine both weights and activations, balancing model size reduction and computational efficiency.

c) Weight-only quantization: This type of quantization focuses solely on compressing model weights while keeping activations in full precision. GPTQ [51] and TEQ [52] apply 3- to 4-bit quantization using second-order error approximations (e.g., Hessian diagonal) to minimize quantization error. QuIP [53] reduces weight correlation through preprocessing, improving quantization robustness. FineQuant [54]

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

and Z-FOLD [55] propose fine-grained and mixed-precision quantization to better adapt to local weight distributions across layers. SignRound [56] enhances rounding by aligning quantized weights with their signs and scales. AFPQ [57] further improves asymmetric quantization through adaptive scaling and outlier suppression. These approaches tackle major challenges such as rounding error, outlier sensitivity, and correlation between weights. In addition, methods like LLM-Int8 [58] and OWQ [59] introduce group-wise quantization or local clustering to manage precision loss more effectively. AdaDim [60], AWQ [61], and SmoothQuant [45] focus on reducing quantization error by adjusting weight scaling dynamically or preconditioning the model structure. SPQR [62] and LLM-MQ [63] also adopt various forms of quantization-aware matrix factorization to optimize memory-accuracy tradeoffs.

d) Weight and activation quantization: For greater compression, some methods quantize both weights and activations, which introduces additional challenges like dynamic range mismatches and outlier activations. ZeroQuant [64] uses dynamic grouped quantization to partition activations for improved calibration. ZeroQuant-FP [65] switches to FP8 quantization to better handle long-tail activation distributions while maintaining hardware efficiency. Atom [66] and MoFQ [67] utilize mixed-precision quantization for sensitive components like attention layers and layer norm, whereas FPTQ [68] focuses on tuning quantization ranges using calibration data. AWEQ [69] and SmoothQuant [45] propose prescaling of activations to align with quantization ranges, which effectively reduces the impact of extreme values. Outlier Suppression [70] introduces filtering mechanisms to prevent rare but large activations from dominating scaling factors.

e) Efficient fine-tuning and quantization of parameters: Parameter-efficient fine-tuning is one of the most promising directions for reducing memory usage while preserving model performance. QLoRA [71] introduces low-rank adaptation combined with 4-bit quantization, enabling scalable fine-tuning of large models. LoftQ [72] and QaLoRA [73] enhance this by decoupling quantization from fine-tuning, minimizing interference between the two processes. LqLoRA [74] and IRQ-LoRA [75] further refine quantization-aware LoRA tuning by dynamically assigning precision to critical modules. Other approaches like PEFT [76], AlphaTuning [77], and QFT [78] introduce sparsity- or rank-aware learning objectives to optimize adaptation quality while operating under quantized constraints.

f) Discussion: QAT simulates quantization during training to reduce performance degradation, with methods like BitDistiller and LLM-QAT using distillation to lower data requirements. However, QAT remains resource-intensive. PTQ offers a lighter alternative by applying quantization post-training, supporting both weight-only and weight-activation schemes. While easier to deploy, PTQ may suffer under low-bit settings due to outliers and a lack of gradient guidance. Future work should focus on better calibration, hardware-aware formats, and hybrid strategies that adapt per layer.

E. Low-Rank Matrix Decomposition

1) Problem Statement: LRMD simplifies models by breaking down weight matrices into smaller ones, reducing size and computational load. To illustrate LRMD clearly, we take KnGPT2 [79] as an example, and provide a definition as follows:

$$\left(\hat{\mathbf{A}}, \hat{\mathbf{B}}\right) = \underset{(\mathbf{A},\mathbf{B})}{\operatorname{argmin}} \|\mathbf{W} - \mathbf{A} \otimes \mathbf{B}\|_2^2 \tag{16}$$

where A denotes a matrix $\in \mathbf{R}^{m_1 \times n_1}$, B denotes a matrix $\in \mathbf{R}^{m_2 \times n_2}$, and $\mathbf{A} \otimes \mathbf{B}$ denotes a block matrix $\in \mathbf{R}^{m \times n}$. $\hat{A}$ and $\hat{B}$ are Kronecker factors estimated from the corresponding weight matrix W. By the above equation, the optimal solution can be obtained by rank-1 singular value decomposition (SVD) approximation of W.

2) Taxonomy: We categorize low-rank decomposition techniques into three main types: Kronecker factorization, SVD, and tensor train decomposition (TTD), based on their decomposition schemes and application targets within LLMs.

a) Kronecker factorization: This method approximates large weight matrices (e.g., in attention or MLP layers) by Kronecker products of smaller matrices, significantly reducing parameter counts and memory usage. For example, KNGPT2 [79] applies Kronecker decomposition to attention weights, while LoRDF [80] and LoSparse [81] extend this idea to multiple transformer components. LPLR [82] further explores dynamic Kronecker ranks for adaptivity. These methods excel in modeling structured sparsity and are particularly hardware-friendly due to their regular memory access patterns.

b) SVD-based methods: These approaches decompose weight matrices (typically in linear and attention layers) into low-rank components using truncated SVD. SVD-LLM [83] inserts low-rank linear modules into the transformer layers, while LASER [84] applies SVD to attention and feed-forward networks, maintaining performance with minimal fine-tuning. tSVD [85] extends this by analyzing rank sensitivity across layers to guide decomposition granularity. Compared to Kronecker methods, SVD offers more flexible rank control and is well-suited for posttraining compression.

c) Tensor train decomposition: TensorGPT [86] employs TTD to compress the large embedding matrices by reshaping them into high-order tensors and applying tensor train factorization. This technique is particularly effective in embedding-heavy models or multilingual settings, offering large memory savings with minimal accuracy drop. Unlike matrix-based methods, TTD excels in compressing very large parameter blocks where spatial redundancy exists.

d) Comparison: Kronecker methods are efficient and regular but may struggle with unstructured redundancy. SVD methods offer greater flexibility and have been widely used for fine-tuning efficiency. Tensor decomposition methods like TTD achieve superior compression on extremely large tensors (e.g., embeddings) but may introduce complexity in implementation and require more tuning. Future work may explore hybrid decomposition schemes that adapt per module based on tensor shape and sensitivity.

e) Discussion: LRMD compresses LLMs efficiently by approximating large matrices with low-rank forms. SVD-based methods (e.g., SVD-LLM and TSVD) suit linear and attention

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

layers for fast deployment, while Kronecker-based approaches (e.g., LoRD and LPLR) offer structured compression and hardware efficiency. Tensor-based methods like TensorGPT target embeddings and are effective on edge devices (e.g., Raspberry Pi 5). However, these methods are rarely tested in real-world applications. Future work should explore domain-specific deployments and improve adaptive rank selection and hardware alignment to enhance practical utility.

III. Efficient LLM Architecture

A. Efficient Attention

The success of LLM can be attributed to the strong ability of the transformer in terms of feature extraction, hence, designing efficient LLM architecture is equal to designing efficient transformer architecture. Based on the observation with respect to cost analysis of the LLMs aforementioned, we can conclude that efficient attention mechanisms (i.e., sparse attention) are a promising solution tailored for efficient LLM architecture, which can make mobile devices possess LLM ability.

1) Sparse Attention: As shown in Fig. 6b, sparse attention aims to create a sparse attention matrix to minimize the number of dot products. These methods fall into two categories: decomposition-based and learning-based approaches. Decomposition-based methods break down the entire self-attention computation into quicker operations using sparse factorization or similar techniques. These operations are then combined to approximate dense attention operations. On the other hand, learning-based methods tackle attention complexity by incorporating trainable sorting mechanisms, hash functions, predictive sparse attention mechanisms, and other strategies to learn sparse attention outputs.

1) **Decomposition-Based Methods:** They effectively reduce self-attention complexity while preserving performance. Approaches such as Longformer [190] and BigBird [87] optimize attention mechanisms specifically for long-sequence processing, which is essential for handling large datasets. These methods enable hardware acceleration, thereby improving overall efficiency. However, their complexity may pose challenges in implementation, and careful tuning is often necessary to achieve optimal results. In addition, while they enhance scalability, there can be tradeoffs in accuracy depending on the specific decomposition techniques used [88], [89], [90], [91].
2) **Learning-Based Methods:** They leverage sparse attention mechanisms to further reduce self-attention complexity. For instance, sparse sinkhorn attention [92] reduces memory complexity through learnable sorting networks, enhancing flexibility. SAC [93] dynamically optimizes attention edges, improving computational efficiency. HEPOS [94] assigns attention heads to specific input subsets, further decreasing computational costs. Although these methods show promise, they may require extensive training data and can introduce variability in performance. In addition, ensuring robustness in diverse scenarios remains a challenge. Overall, learning-based methods offer innovative solutions, yet they necessitate careful consideration of their limitations.

2) Approximate Attention: Approximate attention methods [as shown in Fig. 6(c)] present a promising approach to streamline computational demands and bolster efficiency, especially when dealing with longer sequences. These methods can be classified into two distinct categories. The first category uses mathematical strategies to approximate the attention mechanism, thereby curtailing computational and storage overheads. Conversely, the second category entails structural adjustments within the model to achieve attention approximation, consequently enhancing overall efficiency.

1) **Methods Based on Mathematical Strategies:** They focus on reducing attention complexity through kernel-based, low-rank, and approximation techniques. Kernel-based self-attention [95] reduces complexity via kernel feature mappings, while Linformer [96] leverages low-rank approximations of attention matrices to save memory. Efficient Attention [97] employs double softmax for linear attention approximation, and Performers [98] use orthogonal random feature compression for unbiased attention estimation. Other methods [99], [100], [101], [102] also obtain superior performance.
2) **Methods Based on Model Architecture:**They focus on reducing computational complexity and enhancing performance through architectural innovations. Reformer [191] reduces complexity with locality-sensitive hashing and reversible layers, cutting memory usage. CrossViT [103] combines cross-attention across different-sized patches for more effective visual feature extraction, lowering computational costs. Nystromformer [104] approximates self-attention to reduce computational load, while deformable attention [105] improves adaptability by relocating key points flexibly. Mega [106] addresses weak induction bias and complexity with a gate-controlled moving average mechanism, while [107] proposes a layer with linear complexity via secondary attention.

3) Discussion: Efficient attention mechanisms are key to reducing the computational complexity of transformer models. Sparse attention methods, like Longformer and BigBird, focus on relevant input portions to handle long sequences efficiently but may struggle with generalization. Predictive methods, such as routing transformer and FSAT, optimize attention but introduce training complexities. Approximate techniques, including kernel-based approaches (e.g., Performers) and low-rank approximations (e.g., Linformer), further reduce complexity at the cost of some accuracy. Restructured mechanisms like reformer and deformable attention improve efficiency but can complicate implementation. Each method balances computational efficiency and performance differently.

B. Hardware Optimized Attention

The attention mechanism is a primary computational and memory bottleneck during the inference phase of LLMs. To address this, recent research has focused on hardware-friendly optimizations that aim to reduce memory consumption, minimize communication overhead, and enhance throughput and energy efficiency—all crucial for deploying LLMs in

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

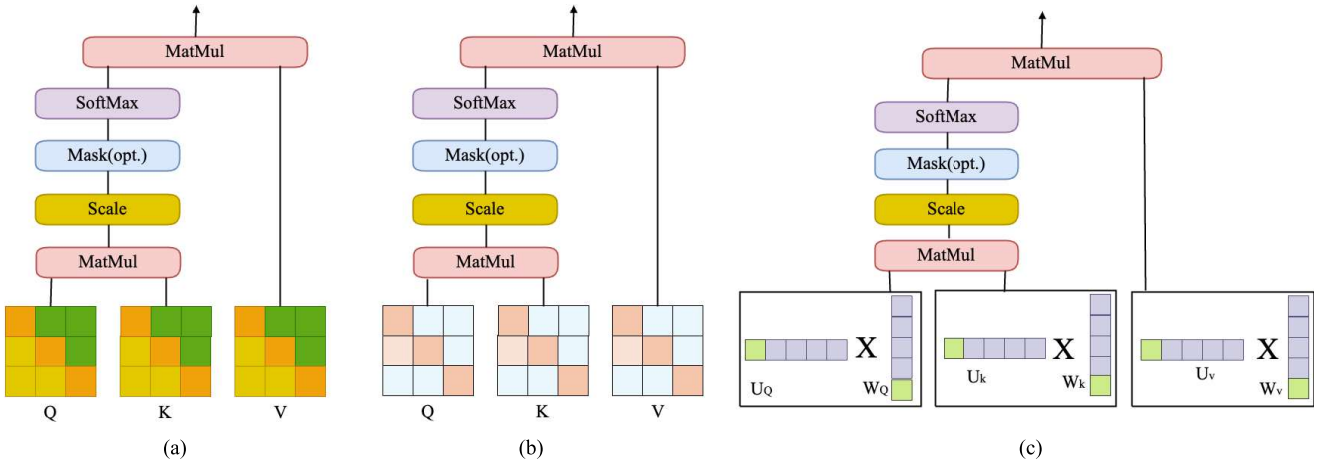


Fig. 6. Comparison of various attention methods. (a) Self-attention. (b) Sparse attention. (c) Approximate attention.

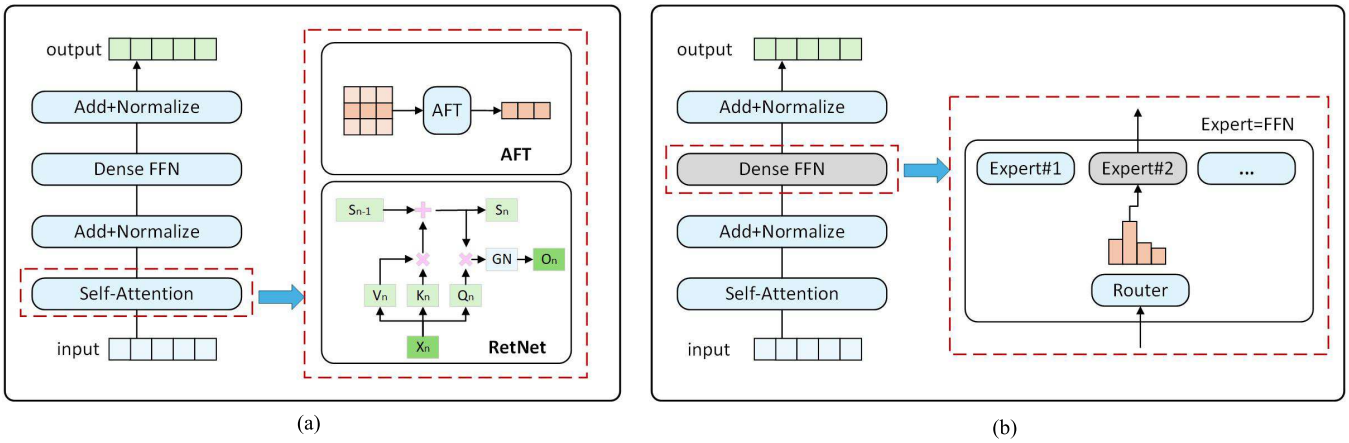


Fig. 7. Comparison between attention-free and MOE. (a) Mechanism of attention-free in LLM. (b) Mechanism of MOE in LLM.

latency-sensitive or resource-constrained environments. Several representative methods tackle these challenges from different perspectives. For example, A3 [108] introduces a candidate key selection mechanism that prunes unnecessary attention computations, along with a parallel-friendly hardware pipeline. It achieves up to 3.25× speedup and 4.19× energy savings on edge devices. In addition, FlashAttention and FlashAttention-2 [109], [110] fuse all attention operations into a single GPU kernel and optimize memory access for SRAM, which reduces off-chip memory usage. These methods yield 2–4× acceleration and significantly lower memory usage, especially on long sequences. PagedAttention [111] proposes a paged key-value cache management strategy to eliminate memory fragmentation and support high-throughput, concurrent inference requests. Other methods, such as LightSeq [112] and ELSA [113], improve execution efficiency through kernel fusion, sparsity-aware techniques, and heterogeneous memory systems.

1) Discussion: Hardware-optimized attention mechanisms have shown significant improvements in efficiency, reducing FLOPs (e.g., 2.7× in FlashAttention), latency, and energy consumption (e.g., 4.19× savings in A3). These methods address key challenges such as memory fragmentation, kernel fusion, and communication bottlenecks. However, limitations persist, including the complexity of algorithm–hardware co-design, the lack of generalization across architectures, and the risks of accuracy loss under aggressive approximation. Moreover, the absence of standardized benchmarks hampers fair evaluation. Future work should focus on developing generalizable frameworks and unified metrics to guide scalable, robust deployment.

C. No Attention

To improve the efficiency of LLMs, many studies have adopted attention-free mechanisms [as shown in Fig. 7(a)] to optimize LLMs. These approaches fall into two categories: state space models and other model structures.

1) Methods Based on State Space Models: The structured state space (S4) model [114] revolutionizes state space models by combining low-rank and regular components through a simplified generating function in frequency space, resulting in a robust Cauchy kernel. In contrast, the gated state space (GSS) model [115] employs gated units to streamline operations. It achieves 2–3× faster training speeds compared to S4 and diagonalized state space (DSS) models [116] while maintaining competitive performance on various benchmarks.

Recent advancements also include the integration of state space models with attention mechanisms. Sparse modular

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

activation (SMA) [117] dynamically activates attention submodules, enhancing efficiency in neural networks, while the block-state transformer (BST) [118] combines SSMs with local attention for improved long-sequence processing. Other models, such as *H*3 [119] and Mamba [120], have shown impressive results in enhancing sequence memory and scaling with sequence length, respectively. These innovations collectively demonstrate significant advancements in the efficiency and effectiveness of state space models and their applications.

2) Methods Based on Other Model Structures: The attention-free transformer (AFT) [121] eliminates traditional attention mechanisms by combining keys and values with learned positional biases, performing element-wise multiplication between queries and a reduced context. This approach retains the benefits of direct interactions, making it an efficient alternative. In contrast, the recurrent memory transformer (RMT) [122] integrates a memory mechanism with special tokens to carry memory states across segments, enabling recurrent properties and removing input length limitations.

In addition, models like megabyte [123], hyena [124], linear recurrent unit (LRU) [125], RWKV [126], and RetNet [192] showcase innovative strategies for improving sequence modeling and inference efficiency. Megabyte's block-based approach and hyena's long convolutions enhance the handling of long sequences, while LRU and RWKV optimize recurrent neural network architectures for better long-range dependency capture. RetNet further enhances performance by replacing multihead attention with a multiscale preservation mechanism. Hence, these methods highlight significant progress in developing efficient alternatives to traditional models.

a) Discussion: The evolution of state space models and alternative architectures to traditional transformers showcases significant advancements in sequence modeling efficiency. Models like the S4 and the GSS effectively optimize computational resources and speed, yet they may introduce complexities in implementation and dependence on specialized hardware. Conversely, approaches like the AFT and the RMT simplify operations and eliminate some limitations of traditional attention mechanisms, though they might compromise flexibility and expressive power.

D. MOE

The mixture of experts (MoEs) [127] model [as shown in Fig. 7(b)] is a powerful technique in optimizing LLMs. By utilizing specialized models, or "experts" that focus on distinct subtasks, MoE enhances the model's capacity without increasing computational load. During inference, it selectively activates a subset of experts, reducing computational and memory overhead. Notably, [193] showcases sparse MoE models with up to 1.1 trillion parameters, demonstrating competitive performance in zero-shot, highlighting their potential to replace traditional dense architectures.

The MoEs framework has led to significant advancements in LLMs. For instance, GShard [128] enables training a multilingual transformer model with over 600 billion parameters through sparse-gate MoE, streamlining parallel computation. Meanwhile, V-MoE [129] enhances the vision transformer (ViT) by replacing dense feed-forward layers with sparse experts, allowing for a balance between performance and inference costs by adjusting sparsity levels.

Other notable models, such as LIMoE [130] and GLaM [131], demonstrate the effectiveness of integrating sparse MoE in multimodal contexts and efficient inference, respectively. LIMoE outperforms CLIP in various tasks, while GLaM, despite having 1.2 trillion parameters, requires only half the FLOPs for inference compared to denser models. These innovations reflect the potential of MoE architectures to achieve higher performance while maintaining computational efficiency, paving the way for further enhancements in LLMs.

1) Discussion: MoEs models offer a promising path for scaling LLMs efficiently by activating only a small subset of expert modules per input, thereby increasing parameter capacity without proportional inference cost. Methods like GShard and GLaM illustrate notable FLOPs reductions while maintaining strong downstream performance. However, these gains come with tradeoffs: routing algorithms add training instability and latency, expert load imbalance can reduce hardware utilization, and sparsity may hinder generalization across diverse tasks. Addressing these challenges—through improved routing, expert balancing, and unified optimization strategies—remains critical for making MoE architectures robust and widely deployable in real-world applications.

IV. Inference Acceleration

Although compression techniques (e.g., pruning, quantization, distillation, and low-rank decomposition) reduce model size and often improve inference speed, their impact on accuracy can be uncertain. In contrast, nonmodel-level methods (as shown in Fig. 1)—such as early exit, KV-cache optimization, and decoding strategies—do not alter model structure and mainly reduce memory or I/O cost, which may not lead to actual speedup in computation. Nevertheless, they remain active research directions due to their practicality. Token pruning, as an exception, can reduce both memory and computation by skipping less important tokens. In this section, we analyze these inference acceleration methods in detail.

A. Early Exit

Early exiting (as presented in Fig. 8) is a strategy designed to reduce computational expenses and improve response time by terminating the inference process prematurely in LLMs. These methods can be categorized into two main groups: the first group primarily concentrates on speeding up the inference process through early exiting, whereas the second group is centered on enhancing and optimizing the performance and efficiency of early exit techniques.

1) Early Exit Strategies: Enhance efficiency by allowing the model to terminate processing based on input complexity. One approach is the multiexit architecture [132], where early exit branches are inserted after intermediate layers. This method is especially beneficial in resource-constrained environments and offers several designs tailored for visual transformer networks, facilitating quick predictions. Another notable method is the CALM approach [133], which uses a scoring mechanism to assign confidence scores at each layer, enabling consistent early exits based on these scores.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

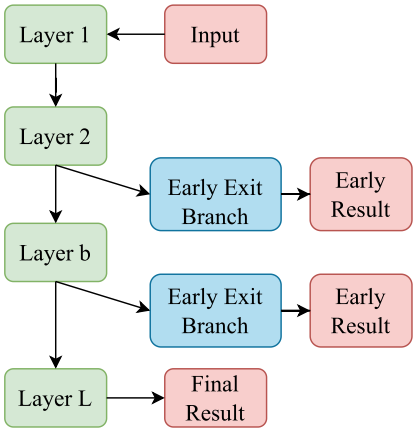


Fig. 8. Mechanism of early exit in LLM compression proposed by method [132].

Other studies, such as SkipDecode [134], focus on token-level exits by establishing uniform exit points within sequences, thereby balancing computational efficiency with performance. Similarly, ConsistentEE [135] employs reinforcement learning to decide exit points dynamically. These advancements demonstrate the potential of early exit strategies to improve model efficiency while maintaining accuracy across various applications.

2) Optimizing Early Exit Strategies: Enhances efficiency without compromising performance. One effective method is the FREE framework [136], which introduces a shallow-deep module that aligns the decoding of current tokens with previously processed early exit tokens. This synchronization accelerates performance in larger models and reduces the computational burden associated with determining exit thresholds.

Another approach, M4 [137], employs a multipath task execution framework that supports flexible fine-tuning and enables block-wise execution. It supports partial activation, adapting to various task requirements while maintaining a lightweight structure. Overall, these optimizations demonstrate promising improvements in model efficiency and effectiveness, making early exit strategies more viable across different applications.

a) Discussion: Early exit strategies are most effective when processing easy samples or performing relatively simple tasks. For instance, in classification tasks like sentiment analysis or topic detection, if the model becomes confident about its prediction after only a few layers, it can exit early—saving computation without sacrificing accuracy. This approach works particularly well when most inputs do not require deep reasoning or long-range context. On the other hand, for complex tasks such as summarization or code generation, which often require the full depth of the model to understand context, early exits are less effective.

B. KV-Cache Optimization

The KV cache [138] (as shown in Fig. 9) stores keys and values throughout the inference process, which consumes considerable memory space in large models and long sequences. Optimizing KV cache memory is crucial for enhancing inference efficiency.

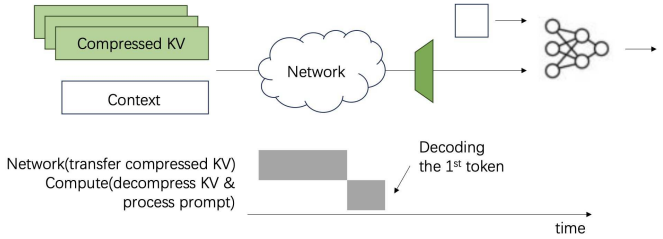


Fig. 9. Mechanism of KV cache proposed by method [194].

1) Specific Algorithm: The study [139] proposes the landmark method, employing special tokens to manage KV cache storage. It partitions input into fixed-length blocks and enables the model to retrieve and incorporate previous blocks during inference using an attention allocation mechanism. This approach preserves attention mechanism flexibility while significantly cutting down on computation and memory usage. Drawing on the persistence of importance hypothesis, scissorhands [140] selectively retains critical key-value pairs, reducing KV cache memory usage by up to five times without sacrificing model quality. The work [141] introduces the dynamic context pruning method, which employs a trainable mechanism to identify and eliminate noninformative KV cache tokens, leading to substantial reductions in memory usage and computational demands during inference. Heavy-Hitter Oracle [142] optimizes KV cache utilization by identifying essential "heavy-hitters," which are pivotal tokens for the generation process. Through a KV cache eviction strategy, this method effectively reduces KV cache size and inference latency while maintaining high-quality generation outputs.

a) Discussion: Combining the aforementioned approaches, optimizing KV cache management in large models and long sequences highlights the critical importance of conserving memory space during inference. These methods, such as random access, dynamic context pruning, and the persistence of importance hypothesis, effectively reduce memory footprint and computational demands in inference processes. They maintain flexibility in attention mechanisms while enhancing inference efficiency and preserving generation quality.

C. Token Pruning

Token pruning (see Fig. 10) improves LLM inference by removing less important tokens from the input sequence. This reduces computation and memory usage, especially in long sequences, while maintaining accuracy.

1) For LLMs: In LLMs, pruning decisions are typically based on attention scores. For example, SpAtten [143] removes tokens and attention heads with low attention weights, while LTP [144] uses learnable thresholds to decide which tokens are unimportant. Other methods like Transkimmer and [146] identify and discard tokens that contribute little to the final output.

2) For Vision-Language Models: In vision-language models (VLMs), pruning often involves both text and image tokens. PuMer [147] keeps only the most relevant image tokens based

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

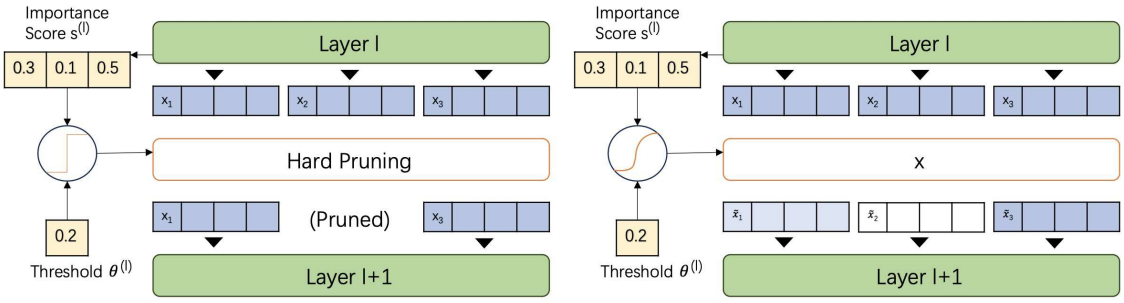


Fig. 10. Mechanism of token pruning in LLM compression proposed by method LIP [144].

on the input text, while SmartTrim [148] uses lightweight modules to remove unnecessary tokens and heads during cross-modal interactions. DynamicViT [149] and others [150], [151], [152] dynamically remove image patches based on their contribution.

a) Discussion: Most token pruning methods rely on attention patterns or learned importance scores to guide pruning. The key challenge is identifying which tokens can be safely removed without harming accuracy. These techniques are especially helpful in long inputs or multimodal settings, making LLMs faster and more efficient for real-world use.

D. Decoding Strategy

Decoding strategies in LLMs generate outputs mainly through speculative decoding, parallel decoding, and retrieval-based decoding, each balancing efficiency and accuracy differently.

1) Speculative Decoding: This method speeds up inference by using a smaller, faster model to draft tokens, then verifying with a larger, accurate model (e.g., [153] and SpS [155]). KD techniques like Distillspec [154] improve alignment between models. Speculative decoding achieves good speedups but depends on the smaller model's draft quality; poor drafts may reduce accuracy.

2) Parallel Decoding: As illustrated in Fig. 11, parallel decoding predicts multiple tokens simultaneously to cut sequential delays. Techniques such as ProphetNet [157], Medusa [158], Look-ahead [159], and EAGLE [160] enable faster generation. While highly efficient on parallel hardware, it requires complex architectures and training.

3) Retrieval-Based Decoding: Retrieval-based methods incorporate relevant external texts to improve factual accuracy and context (e.g., LLMA [161], [162], and CoG [163]). This enhances output quality for knowledge-intensive tasks but requires careful management to maintain relevance and fluency.

a) Discussion: Speculative decoding offers a good speed-accuracy balance when a reliable small model exists, making it suitable for latency-sensitive applications that tolerate some approximation. Parallel decoding delivers the highest throughput but at higher complexity, ideal for large-scale, resource-rich deployments. Retrieval-based decoding improves accuracy in knowledge-driven scenarios but may affect fluency if context retrieval is imperfect. The choice depends on the target application's efficiency, accuracy, and complexity needs.

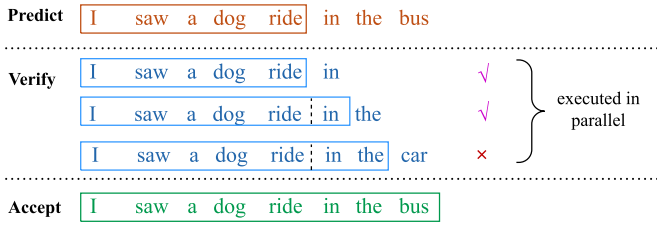


Fig. 11. Parallel-decodingin LLM inference.

E. Inference Acceleration Framework

Against the backdrop of rapid advancements in artificial intelligence technology [195], LLMs have emerged as pivotal engines driving intelligent applications, essential for tackling challenges and enhancing user experiences. In cloud and edge environments, frameworks such as DeepSpeed FastGen, vLLM, and llama.cpp, contribute unique optimization strategies to deliver efficient and cost-effective LLM inference. The framework of inference acceleration is provided in Fig. 12.

1) Cloud Frameworks: Combining vLLM [111], DeepSpeed FastGen [164], and LightLLM [165] creates a powerful cloud-based LLM inference platform. vLLM's centralized scheduler efficiently manages distributed GPU resources, while DeepSpeed FastGen enhances throughput and handles high-concurrency requests. LightLLM's lightweight architecture optimizes memory and speed through advanced techniques like FlashAttention. This integration provides a scalable, flexible, and cost-effective solution for diverse applications, ensuring rapid and efficient inference in cloud environments.

2) Edge Frameworks: Integrating llama.cpp [166], mnn-llm [167], and mllm [168] creates an efficient edge-based LLM inference platform. Llama.cpp's pure C/C++ implementation optimizes performance across diverse architectures, while mnn-llm enables deployment of advanced LLMs on mobile platforms. Mllm's lightweight design and multimodal capabilities ensure fast inference, making it suitable for edge environments. This combination supports a wide range of applications, ensuring scalable, high-performance LLM services accessible across various devices and platforms.

3) Open Source Alternative GPT Model: In NLP, LLMs have excelled in diverse tasks, expanding their reach to multimodal applications. However, prominent models like GPT-4 encounter challenges like substantial size, intensive computa-

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

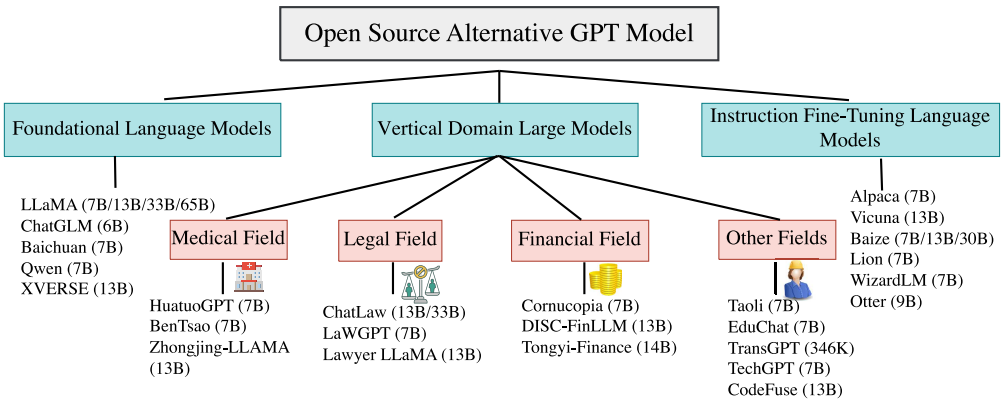


Fig. 12. Overview of open source alternative GPT model.

tional needs, intricate deployment, and closed development. To surmount these hurdles and sustain performance, we explore open-source alternatives to large GPTs. Our emphasis lies on compact, user-friendly models, prioritizing ease of deployment.

The alternative models under investigation meet three key criteria: 1) their relatively small size (around 10 billion parameters) enhances user convenience; 2) they guarantee the capacity to meet fundamental requirements for generating text-based content; and 3) accessible model checkpoints empower researchers for extended exploration and downstream applications. These models are classified into foundational language models, instruction fine-tuning language models, and large models customized for specific vertical domains.

a) Foundational language models: These models predominantly employ nonfine-tuning methods, autonomously designing or optimizing GPT and T5 models. They achieve the complete cycle, incorporating pretraining, supervised fine-tuning, reinforcement learning, and beyond.

The combination of LLaMA, ChatGLM (6B) [169], Baichuan (7B), Qwen (7B) [170], and XVERSE (13B) collectively form a powerful foundation for LLMs tailored for diverse applications. LLaMA and Baichuan excel in general language tasks, while ChatGLM is optimized for bilingual conversational agents. Qwen supports multilanguage capabilities, and XVERSE enhances performance across various languages. In brief, these models enable efficient deployment in commercial applications, and intelligent personal assistants, ensuring high performance and adaptability in a wide range of NLP tasks.

b) Instruction fine-tuning language models: In this context, "Instruction" denotes the articulation of tasks through natural language. For example, in a translation task, add a task directive "Translate the given English utterance to a French utterance" preceding the sentence "I Love You" to be translated, guiding the model on the task and its requirements. By converting various tasks into instructional data, further fine-tuning of language models is performed, enhancing the model's zero-shot performance on unknown tasks.

These language models offer diverse capabilities for fine-tuning and application. Alpaca and Vicuna excel in instruction following, with Vicuna achieving near ChatGPT performance at lower costs. Baize generates high-quality dialogs for multiturn interactions, while Lion enhances instruction handling using adversarial techniques. WizardLM employs evolutionary instruction generation, improving performance significantly. Flan-T5 and LLaVA provide robust multimodal understanding and task performance, and Otter and MultiModal-GPT focus on accurate multimodal instruction. Together, these models cater to varied scenarios, from conversational AI to complex visual-language tasks.

4) Vertical Domain Large Models: Fig. 12 provides an updated and more comprehensive overview of vertical domain LLMs, which are designed and optimized for specific industries such as law, finance, healthcare, and science. Unlike general-purpose LLMs that aim for broad linguistic and world knowledge coverage, vertical models focus on deep understanding and task-specific performance within a well-defined domain. This specialization enables them to provide more accurate, relevant, and trustworthy outputs for domain-specific tasks such as legal reasoning, financial report generation, or clinical diagnosis.

In the legal domain, models like ChatLaw (13B/33B), LaWGPT (7B), and Lawyer LLaMA (13B) are adapted from base architectures such as LLaMA and further trained on legal texts (e.g., statutes, case law, and contracts). These models emphasize interpretability and logical consistency—key requirements in legal reasoning. Some are jurisdiction-specific (e.g., ChatLaw for Chinese law), while others aim for cross-jurisdiction generalization. Key challenges include limited high-quality legal datasets, frequent regulatory changes, and the high reliability demands of legal applications.

In finance, models like Cornucopia (7B), DISK-FinLLM (13B), and Tongyi-Finance (14B) support tasks such as analysis, prediction, sentiment extraction, and risk assessment. Trained on financial news, reports, and market data, these models excel in numerical reasoning and domain-specific language. Trends include multimodal input integration (e.g., time-series + text) and low-latency optimization for real-time applications. Challenges include fast-evolving knowledge, restricted access to proprietary data, and variability across global financial systems.

Overall, vertical domain LLMs are becoming an important subfield in the LLM ecosystem. They represent a shift from

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

TABLE III

EXPERIMENTAL COMPARISON OF LMM COMPRESSION TECHNIQUES (I.E., MODEL QUANTIZATION, MODEL DISTILLATION, MODEL PRUNING, AND LOW RANK MATRIX DECOMPOSITION) IN MAINSTREAM BENCHMARKS. "-" INDICATES THE RESULTS ARE NOT PROVIDED IN THE ORIGIN PAPER

Ref	Base Model	Bit-width	KV Cache	Compression Rate	Param	Perplexity Difference (Wikitext-2)	Perplexity Difference (C4)	Speedup	Category	Year
Model Quantization										
LLM-QAT[46]	LLaMA-30B	4/8	16	-	30B	0.5	0.9	-	QAT	2023
BitDistiller [47]	LLaMA2-13B	2/16	16	-	13B	1.9	-	-	QAT	2024
OneBit[49]	LLaMA-13B	1/16	16	-	175B	4.09	3.64	-	QAT	2022
GPTQ[51]	OPT-175B	3/16	16	-	175B	0.34	0.23	3.24×	Weight-Only Quantization	2022
TEQ[52]	LLaMA-13B	3/32	16	-	13B	0.72	-	-	Weight-Only Quantization	2023
FineQuant[54]	OPT-13B	4/16	16	-	13B	1.3	-	-	Weight-Only Quantization	2023
Z-FOLD[55]	LLaMA-13B	3/16	16	-	13B	0.67	0.60	-	Weight-Only Quantization	2023
AWQ[61]	LLaMA-13B	3/16	16	-	13B	0.43	-	-	Weight-Only Quantization	2023
OWQ[59]	LLaMA-13B	3.1/16	16	-	13B	0.47	-	2.0×	Weight-Only Quantization	2023
SpQR[62]	LLaMA-13B	3.96/16	16	-	13B	0.13	0.11	-	Weight-Only Quantization	2023
QuIP[53]	LLaMA2-70B	2/16	16	-	13B	3.007	3.228	-	Weight-Only Quantization	2024
LUT-GEMM[196]	LLaMA-65B	3/16	16	-	65B	0.14	-	2.04×	Weight-Only Quantization	2024
SqueezeLLM[197]	LLaMA-13B	3/16	16	-	13B	0.51	0.67	2.4×	Weight-Only Quantization	2024
ZeroQuant[64]	GPT-J-6B	8/8	16	-	6B	0.16	-	3.67×	Weight-Activation Quantization	2022
LLM.int8()[58]	OPT-13B	8/8	16	-	13B	-	0.00	1.22×	Weight-Activation Quantization	2022
SmoothQuant[45]	OPT-175B	8/8	16	-	175B	0.18	-	1.56×	Weight-Activation Quantization	2023
RPTQ[198]	OPT-175B	4/4	16	-	175B	2.26	2.15	-	Weight-Activation Quantization	2023
Olive[199]	BLOOM-7B	4/4	16	-	7B	2.11	2.24	4.5×	Weight-Activation Quantization	2023
LLM-FP4[200]	LLaMA-30B	4/8	16	-	30B	0.18	0.13	-	Weight-Activation Quantization	2023
OminiQuant[201]	LLaMA-13B	3/16	16	-	13B	0.59	0.71	-	Weight-Activation Quantization	2023
QT[202]	OPT-1.3B	8/8	16	-	1.3B	17.74	-	-	Weight-Activation Quantization	2024
WKVQuant[203]	LLaMA-13B	8/8	4	-	13B	0.12	0.14	-	KV Cache Quantization	2024
Model Distillation										
GKD [23]	T5-Small/T5-XL	16/16	-	-	77M	-	-	3.8×	General capability distillation	2024
MiniLLM [24]	LLaMA-7B/LLaMA-13B	16/16	-	-	7B	-	-	-	General capability distillation	2024
Model Pruning										
SparseGPT [21]	OPT-175B	16/16	16	50%	87.5B	-0.14	-	-	Unstructured Pruning	2023
Wanda[20]	LLaMA-13B	16/16	16	50%	7.5B	1.06	-	-	Unstructured Pruning	2024
DSnoT [204]	LLaMA-13B	16/16	16	50%	7.5B	0.99	-	-	Unstructured Pruning	2024
SAMSP [205]	LLaMA-13B	16/16	16	50%	7.5B	0.66	0.76	-	Unstructured Pruning	2024
OWL w. SparseGPT [22]	LLaMA-13B	16/16	16	70%	3.9B	9.46	-	-	Unstructured Pruning	2024
RIA [185]	LLaMA-13B	16/16	16	50%	7.5B	0.99	-	0.98×	Unstructured Pruning	2024
GBDT [206]	LLaMA2-7B	16/16	16	30.52%	4.864B	29.5	-	-	Structured Pruning	2023
LLM-Pruner [19]	LLaMA-13B	16/16	16	20%	10.4B	3.6	-	-	Structured Pruning	2023
SliceGPT [207]	LLaMA2-70B	16/16	16	30%	49B	1.73	-	1.87×	Structured Pruning	2024
SparseGPT [21]	OPT-175B	16/16	16	2:4	87.5B	0.39	-	2×	Semi-Structured Pruning	2023
E-sparse [184]	LLaMA-13B	16/16	16	4:8	7.5B	0.58	-	1.53×	Semi-Structured Pruning	2024
RIA [185]	LLaMA-13B	16/16	16	2:4	7.5B	3.32	-	1.63×	Semi-structured Pruning	2024
Wanda[20]	LLaMA-65B	16/16	16	2:4	7.5B	2.69	-	1.24×	Semi-Structured Pruning	2024
Low Rank Matrix Decomposition										
KnGPT2 [79]	GPT-2-S	16/16	-	-	83M	-	-	-	-	2021
TensorGPT [86]	GPT-2-S	16/16	-	-	86M	-	-	-	-	2023
ASVD [208]	LLaMA-7B	16/16	-	-	5.6B	5.46	8.59	-	-	2023
TSVD [85]	OPT-6.5B	16/16	-	-	0.22B	0.24	0.42	4.64×	-	2023
SVD-LLM [83]	LLaMA-7B	16/16	-	-	5.6B	2.26	8.50	-	-	2024

general intelligence to task-specific, high-precision language understanding. Fig. 12 categorizes these models by domain and scale, highlighting both their rapid development and the growing demand for domain-aware intelligence. While they offer promising performance improvements in their respective fields, challenges such as data scarcity, knowledge updating, and deployment under domain constraints remain open issues. As such, the design and implementation of vertical LLMs require not only domain expertise but also careful balancing of performance, reliability, and real-world applicability.

a) Discussion: Open-source alternatives to large GPT models are emerging as efficient, domain-adaptable solutions. Foundational models like LLaMA and ChatGLM achieve strong performance with fewer parameters by leveraging optimized training and data. Instruction-tuned models such as Alpaca and Vicuna further improve generalization and task alignment, particularly in zero-shot scenarios. Meanwhile, vertical domain models—targeting fields like healthcare, law, and finance—embed domain-specific knowledge to deliver more accurate and context-aware outputs. A key trend is the shift toward lightweight, fine-tuned models tailored for real-world needs, reflecting growing interest in accessible, efficient LLMs for specialized and general-purpose use cases.

V. VALIDATION

Many studies propose LLaMA-based LLM compression methods to reduce model size and complexity for resource-limited deployment. For a fair and concise comparison, we report representative results from existing methods on the Wikitext-2 (83 GB) and CA (783 GB) datasets.

As depicted in Table III, we provide a comprehensive comparison of LLM compression techniques (i.e., model quantization, model distillation, model pruning, and low rank matrix decomposition) in mainstream benchmarks in terms of "Bit-width," "KV Cache," "Compression Rate," "Param," "Perplexity Difference," "Speedup," "Category," and "Year." Table III can yield a series of conclusions as follows. 1) LLaMA and its variants are representative open-source foundation models in the field of LLMs. Nearly all advancements in LLM compression algorithms are developed and rigorously evaluated using LLaMA and its variants, owing to their fully open-sourced codebase, excellent performance, and ease of deployment. We can obtain a critical insight: while closed-source models (i.e., ChatGPT and GPT4) achieve milestones of remarkable performance, those models significantly hinder the progress of the LLM community due to their closed-source. In contrast, the open-source models (i.e., LLaMA and its variants) play as a catalyst, fostering the development of LLM inference acceleration and enabling worldwide researchers to benefit from open sources. Looking into the future, it is expected that more models following the open-source paradigm of LLaMA, such as DeepSeekV3 [209], will emerge, further facilitating innovation in LLM compression, and efficiency optimization. 2) We can clearly observe that the methods achieve a better tradeoff between complexity and performance, significantly enhancing the inference efficiency

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

of LLMs. Specifically, the Olive achieves a 4.5× than baseline (BLOOM-7B) in terms of speedup, demonstrating the strong potential of low-bit quantization techniques in accelerating LLM inference. In addition, TSVD obtains a 4.64× acceleration, highlighting the effectiveness of LRMD in LLM compression. Model distillation, and model pruning can draw the same conclusion.

In summary, these experimental results play a pivotal role in advancing the field of lightweight LLM, enabling more efficient deployment and broader application of LLMs in resource-limited scenarios.

VI. Applications

A. Autonomous Driving and Video Processing

Vision-language applications such as autonomous driving and video understanding have been demonstrating the potential of VLMs and MMLMs in real-world scenarios. For example, EM-VLM4AD [171] addresses real-time inference challenges in autonomous driving by designing a lightweight multiframe VLM, enabling efficient visual question answering (VQA) with reduced resource demands [210]. In video understanding, SlotVLM [172] proposes a token decomposition approach to reduce input size and improve reasoning. However, their inference efficiency primarily depends on visual processing modules, which complicates the isolation of gains achieved through LLM acceleration.

B. Image Processing

VLMs have enhanced LLMs for VQA tasks [211], yet they often struggle with text-rich images. Traditional methods typically use fixed query embeddings, which can be inadequate for images with substantial text due to token limitations. To improve this, the study [173] introduced BLIVA, an enhanced version of InstructBLIP. BLIVA combines learned query embeddings with direct projections of encoded image patches, significantly boosting performance. It achieved a 17.76% increase on the OCR-VQA benchmark and a 17.72% improvement over InstructBLIP on the Multimodal LLM benchmark. This research emphasizes the potential of integrating visual information to advance image processing.

C. LLM Agent Applications

LLM agents, such as those used in robotics, web automation, and code generation, demand fast and frequent inference to enable real-time decision-making. Unlike vision-language tasks, where latency is often dominated by visual encoders, the performance bottleneck in LLM agents lies in the model inference itself. Recent works like RAP [174] and D-PRUNER [175] show that pruning can significantly reduce inference time and memory usage without sacrificing performance. These examples highlight that inference acceleration is not just beneficial but essential for deploying responsive and efficient LLM agents in practical settings.

D. AI Interaction Models

In the U.S., approximately 33 infants are born daily with irreversible hearing loss, often to hearing parents unfamiliar with American sign language (ASL), leading to language deprivation syndrome. To address this, multimodal robots provide interactive learning environments that enhance ASL education and foster inclusive communication. The study [176] developed a lightweight ASL recognition model for the humanoid robot Pepper, enabling efficient sign recognition in resource-constrained settings. By integrating LLMs, Pepper can generate natural co-speech gestures. This approach creates a socially-aware AI interaction model, improving human–robot dialogs and laying the groundwork for future educational applications.

VII. Open Issues and Trends

To better reflect the multifaceted challenges in LLM inference acceleration, we categorize the open issues into two distinct perspectives: the Academic Perspective, which emphasizes theoretical soundness, algorithmic innovation, and long-term exploration; and the Industrial Perspective, which focuses on deployability, resource constraints, and real-world utility. This categorization helps clarify differing priorities and constraints in research and deployment settings.

A. Academic Perspective

1) Balancing Compression Ratio and Performance: In academia, compression techniques (e.g., pruning, quantization, distillation) aim to explore the limits of model compression without significant performance degradation. However, achieving an optimal tradeoff between compression ratio and task performance remains an unsolved problem. Research continues to explore dynamic, data-aware compression strategies that can adaptively balance size and accuracy.

2) Generalization Capability of Lightweight Models: Reducing model size can impair its generalization to unseen distributions or downstream tasks. Academic research is tasked with understanding the theoretical underpinnings of why certain lightweight techniques cause degradation, and how to design new strategies (e.g., structured sparsity or low-rank adaptation) that preserve or even improve generalization despite fewer parameters.

3) Interpretability and Transparency: There is growing academic interest in making compressed or accelerated models more interpretable. Lightweight models, due to their simpler structure, provide a potential path toward understanding model decisions. However, new methods are needed to visualize and analyze decision processes in real time, especially under resource constraints.

B. Industrial Perspective

1) Reasoning Models Acceleration: Recent advances in LLMs have led to the emergence of powerful reasoning-centric models such as DeepSeek-R1 [212], Grok-4 [213], and GPT-4o [214]. These models exhibit enhanced logical reasoning and multimodal understanding capabilities, setting

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

new performance benchmarks across a range of tasks. However, their superior abilities often come at the cost of extreme computational overhead, with architectures relying heavily on large-scale parameters and MoEs designs. While such structures improve training efficiency, they introduce significant inference-time challenges, especially for real-time or resource-constrained deployment scenarios. As a result, how to accelerate the inference of these next-generation reasoning models—without sacrificing their core capabilities—has become an urgent and important research problem. Addressing this challenge is critical for the practical use of reasoning LLMs in latency-sensitive applications.

2) Adaptation to Heterogeneous Hardware: In production, LLMs must run efficiently across CPUs, GPUs, TPUs, and edge devices with varied memory and compute capabilities. Techniques like operator fusion, hardware-aware quantization, and compiler optimization (e.g., TensorRT and XLA) are critical. However, developing unified deployment frameworks and generalizable compression methods remains an industrial priority.

3) Privacy and Security Concerns: Industrially deployed models often handle user-sensitive data. Lightweight models may be more susceptible to model inversion and data leakage. Thus, real-world systems must integrate privacy-preserving inference, such as encrypted model execution, differential privacy, or secure federated inference, to ensure compliance and trustworthiness.

VIII. Conclusion

LLMs have achieved significant success in both industrial and academic domains; however, their increasing size and complexity lead to high computational costs and decreased deployment efficiency. This survey analyzes LLM compression from multiple perspectives: foundational principles, algorithms, applications, and open issues. We begin by introducing the key principles of LLM compression and categorizing state-of-the-art algorithms based on their approaches to reducing model complexity. Following the lifecycle of LLMs, we systematically examine compression techniques that span model training, optimization, and deployment, presenting a new taxonomy to guide future efforts in this area. In addition, we explore significant applications of compressed LLMs, identify critical research challenges, and suggest promising directions for future investigation. Through this comprehensive review of recent advancements in LLM compression, we aim to provide valuable insights and guidelines for researchers and practitioners in the field.

References

[1] (2022). *ChatGPT*. [Online]. Available: https://openai.com/blog/chatgpt

[2] (2023). *Claude*. [Online]. Available: https://www.anthropic.com/index/claude-2-1

[3] A. Hendy et al., "How good are GPT models at machine translation? A comprehensive evaluation," 2023, *arXiv:2302.09210*.

[4] X. Yang, Y. Li, X. Zhang, H. Chen, and W. Cheng, "Exploring the limits of ChatGPT for query or aspect-based text summarization," 2023, *arXiv:2302.08081*.

[5] V. Liévin, C. E. Hother, A. G. Motzfeldt, and O. Winther, "Can large language models reason about medical questions?," *Patterns*, vol. 5, no. 3, Mar. 2024, Art. no. 100943.

[6] Z. Ma, Y. Mei, and Z. Su, "Understanding the benefits and challenges of using large language model-based conversational agents for mental well-being support," in *Proc. AMIA Annu. Symp.*, 2023, p. 1105.

[7] T. B. Brown et al., "Language models are few-shot learners," in *Proc. NIPS*, 2020, pp. 1877–1901.

[8] A. S. Luccioni, S. Viguier, and A. Ligozat, "Estimating the carbon footprint of Bloom, a 176B parameter language model," *J. Mach. Learn. Res.*, vol. 24, no. 253, pp. 1–15, 2022.

[9] L. Ma et al., "Pareto-wise ranking classifier for multiobjective evolutionary neural architecture search," *IEEE Trans. Evol. Comput.*, vol. 28, no. 3, pp. 570–581, Jun. 2024.

[10] L. Ma, H. Kang, G. Yu, Q. Li, and Q. He, "Single-domain generalized predictor for neural architecture search system," *IEEE Trans. Comput.*, vol. 73, no. 5, pp. 1400–1413, May 2024.

[11] L. Ma et al., "A novel fuzzy neural network architecture search framework for defect recognition with uncertainties," *IEEE Trans. Fuzzy Syst.*, vol. 32, no. 5, pp. 3274–3285, May 2024.

[12] N. Li, L. Ma, G. Yu, B. Xue, M. Zhang, and Y. Jin, "Survey on evolutionary deep learning: Principles, algorithms, applications, and open issues," *ACM Comput. Surveys*, vol. 56, no. 2, pp. 1–34, Feb. 2024.

[13] L. Ma et al., "Learning to optimize: Reference vector reinforcement learning adaption to constrained many-objective optimization of industrial copper burdening system," *IEEE Trans. Cybern.*, vol. 52, no. 12, pp. 12698–12711, Dec. 2022.

[14] H. Naveed et al., "A comprehensive overview of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 5, pp. 1–72, 2025.

[15] Z. Wan et al., "Efficient large language models: A survey," 2023, *arXiv:2312.03863*.

[16] M. Xu et al., "A survey of resource-efficient LLM and multimodal foundation models," 2024, *arXiv:2401.08092*.

[17] W. Wang et al., "Model compression and efficient inference for large language models: A survey," 2024, *arXiv:2402.09748*.

[18] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A survey on model compression for large language models," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 1556–1577, Nov. 2024.

[19] X. Ma, G. Fang, and X. Wang, "LLM-pruner: On the structural pruning of large language models," 2023, *arXiv:2305.11627*.

[20] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," 2023, *arXiv:2306.11695*.

[21] E. Frantar and D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 10323–10337.

[22] L. Yin et al., "Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMs to high sparsity," 2023, *arXiv:2310.05175*.

[23] R. Agarwal, N. Vieillard, P. Stanczyk, S. Ramos, M. Geist, and O. Bachem, "GKD: Generalized knowledge distillation for auto-regressive sequence models," 2023, *arXiv:2306.13649*.

[24] Y. Gu, L. Dong, F. Wei, and M. Huang, "MiniLLM: Knowledge distillation of large language models," 2023, *arXiv:2306.08543*.

[25] C. Liang, S. Zuo, Q. Zhang, P. He, W. Chen, and T. Zhao, "Less is more: Task-aware layer-wise distillation for language model compression," in *Proc. 40th Int. Conf. Mach. Learn.*, 2023, pp. 20852–20867.

[26] A. H. Jha, D. Groeneveld, E. Strubell, and I. Beltagy, "Large language model distillation doesn't need a teacher," 2023, *arXiv:2305.14864*.

[27] Y. Huang, Y. Chen, Z. Yu, and K. McKeown, "In-context learning distillation: Transferring few-shot learning ability of pre-trained language models," 2022, *arXiv:2212.10670*.

[28] C. Qin, W. Xia, F. Jiao, and S. Joty, "Improving in-context learning via bidirectional alignment," 2023, *arXiv:2312.17055*.

[29] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 22199–22213.

[30] Y. Fu, H. Peng, L. Ou, A. Sabharwal, and T. Khot, "Specializing smaller language models towards multi-step reasoning," 2023, *arXiv:2301.12726*.

[31] M. Kang, S. Lee, J. Baek, K. Kawaguchi, and S. J. Hwang, "Knowledge-augmented reasoning distillation for small language models in knowledge-intensive tasks," 2023, *arXiv:2305.18395*.

[32] L. H. Li, J. Hessel, Y. Yu, X. Ren, K.-W. Chang, and Y. Choi, "Symbolic chain-of-thought distillation: Small models can also 'think' step-by-step," 2023, *arXiv:2306.14050*.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

[33] C.-Y. Hsieh et al., "Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes," 2023, *arXiv:2305.02301*.

[34] K. Shridhar, A. Stolfo, and M. Sachan, "Distilling reasoning capabilities into smaller language models," in *Proc. Findings Assoc. Comput. Linguistics, ACL*, 2023, pp. 7059–7073.

[35] H. Chen, S. Wu, X. Quan, R. Wang, M. Yan, and J. Zhang, "MCC-KD: Multi-CoT consistent knowledge distillation," 2023, *arXiv:2310.14747*.

[36] N. Ho, L. Schmid, and S.-Y. Yun, "Large language models are reasoning teachers," 2022, *arXiv:2212.10071*.

[37] L. C. Magister, J. Mallinson, J. Adamek, E. Malmi, and A. Severyn, "Teaching small language models to reason," 2022, *arXiv:2212.08410*.

[38] C. Li et al., "Mixed distillation helps smaller language model better reasoning," 2023, *arXiv:2312.10730*.

[39] X. Zhu, B. Qi, K. Zhang, X. Long, Z. Lin, and B. Zhou, "PaD: Program-aided distillation can teach small models reasoning better than chain-of-thought fine-tuning," 2023, *arXiv:2305.13888*.

[40] W. Liu et al., "Mind's mirror: Distilling self-evaluation capability and comprehensive thinking from large language models," 2023, *arXiv:2311.09214*.

[41] Z. Wang et al., "Democratizing reasoning ability: Tailored learning from large language model," 2023, *arXiv:2310.13332*.

[42] C. Cheng, B. Fang, and J. J. Yang, "Distinguishing the correctness of knowledge makes knowledge transfer better," in *Proc. Int. Conf. Artif. Neural Netw.*, 2023, pp. 135–146.

[43] Y. Li et al., "Turning dust into gold: Distilling complex reasoning capabilities from LLMs by leveraging negative data," 2023, *arXiv:2312.12832*.

[44] P. Wang, Z. Wang, Z. Li, Y. Gao, B. Yin, and X. Ren, "SCOTT: Self-consistent chain-of-thought distillation," 2023, *arXiv:2305.01879*.

[45] G. Xiao et al., "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 38087–38099.

[46] Z. Liu et al., "LLM-QAT: Data-free quantization aware training for large language models," 2023, *arXiv:2305.17888*.

[47] D. Du et al., "BitDistiller: Unleashing the potential of sub-4-bit LLMs via self-distillation," 2024, *arXiv:2402.10631*.

[48] H. Wang et al., "BitNet: Scaling 1-bit transformers for large language models," 2023, *arXiv:2310.11453*.

[49] Y. Xu et al., "OneBit: Towards extremely low-bit large language models," 2024, *arXiv:2402.11295*.

[50] Y. Shang, Z. Yuan, Q. Wu, and Z. Dong, "PB-LLM: Partially binarized large language models," 2023, *arXiv:2310.00034*.

[51] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," 2022, *arXiv:2210.17323*.

[52] W. Cheng, Y. Cai, K. Lv, and H. Shen, "TEQ: Trainable equivalent transformation for quantization of LLMs," 2023, *arXiv:2310.10944*.

[53] J. Chee, Y. Cai, V. Kuleshov, and C. De, "QuIP: 2-bit quantization of large language models with guarantees," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–17.

[54] Y. J. Kim, R. Henry, R. Fahim, and H. H. Awadalla, "FineQuant: Unlocking efficiency with fine-grained weight-only quantization for LLMs," 2023, *arXiv:2308.09723*.

[55] Y. Jeon, C. Lee, K. Park, and H.-Y. Kim, "A frustratingly easy post-training quantization scheme for LLMs," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 14446–14461.

[56] W. Cheng et al., "Optimize weight rounding via signed gradient descent for the quantization of LLMs," 2023, *arXiv:2309.05516*.

[57] Y. Zhang et al., "AFPQ: Asymmetric floating point quantization for LLMs," 2023, *arXiv:2311.01792*.

[58] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "GPT3.int8(): 8-bit matrix multiplication for transformers at scale," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 30318–30332.

[59] C. Lee, J. Jin, T. Kim, H. Kim, and E. Park, "OWQ: Lessons learned from activation outliers for weight quantization in large language models," 2023, *arXiv:2306.02272*.

[60] J. H. Heo, J. Kim, B. Kwon, B. Kim, S. J. Kwon, and D. Lee, "Rethinking channel dimensions to isolate outliers for low-bit weight quantization of large language models," 2023, *arXiv:2309.15531*.

[61] J. Lin et al., "AWQ: Activation-aware weight quantization for LLM compression and acceleration," 2023, *arXiv:2306.00978*.

[62] T. Dettmers et al., "SpQR: A sparse-quantized representation for near-lossless LLM weight compression," 2023, *arXiv:2306.03078*.

[63] S. Li et al., "LLM-MQ: Mixed-precision quantization for efficient LLM deployment," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–7.

[64] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 27168–27183.

[65] X. Wu, Z. Yao, and Y. He, "ZeroQuant-FP: A leap forward in LLMs post-training W4A8 quantization using floating-point formats," 2023, *arXiv:2307.09782*.

[66] Y. Zhao et al., "Atom: Low-bit quantization for efficient and accurate LLM serving," 2023, *arXiv:2310.19102*.

[67] Y. Zhang et al., "Integer or floating point? New outlooks for low-bit quantization on large language models," 2023, *arXiv:2305.12356*.

[68] Q. Li et al., "FPTQ: Fine-grained post-training quantization for large language models," 2023, *arXiv:2308.15987*.

[69] B. Li, X. Wang, and H. Xu, "AWEQ: Post-training quantization with activation-weight equalization for large language models," 2023, *arXiv:2311.01305*.

[70] X. Wei et al., "Outlier Suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling," 2023, *arXiv:2304.09145*.

[71] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–9.

[72] Y. Li et al., "LoftQ: LoRA-fine-tuning-aware quantization for large language models," 2023, *arXiv:2310.08659*.

[73] Y. Xu et al., "QA-LoRA: Quantization-aware low-rank adaptation of large language models," 2023, *arXiv:2309.14717*.

[74] H. Guo, P. Greengard, E. P. Xing, and Y. Kim, "LQ-LoRA: Low-rank plus quantized matrix decomposition for efficient language model finetuning," 2023, *arXiv:2311.12023*.

[75] H. Qin et al., "Accurate LoRA-finetuning quantization of LLMs via information retention," 2024, *arXiv:2402.05445*.

[76] J. Kim et al., "Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–8.

[77] S. J. Kwon et al., "AlphaTuning: Quantization-aware parameter-efficient adaptation of large-scale pre-trained language models," 2022, *arXiv:2210.03858*.

[78] Z. Li, X. Liu, B. Zhu, Z. Dong, Q. Gu, and K. Keutzer, "QFT: Quantized full-parameter tuning of LLMs with affordable resources," 2023, *arXiv:2310.07147*.

[79] A. Edalati, M. Tahaei, A. Rashid, V. P. Nia, J. J. Clark, and M. Rezagholizadeh, "Kronecker decomposition for GPT compression," 2021, *arXiv:2110.08152*.

[80] A. Kaushal, T. Vaidhya, and I. Rish, "LORD: Low rank decomposition of monolingual code LLMs for one-shot compression," 2023, *arXiv:2309.14021*.

[81] Y. Li et al., "LoSparse: Structured compression of large language models based on low-rank and sparse approximation," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 20336–20350.

[82] R. Saha, V. Srivastava, and M. Pilancı, "Matrix compression via randomized low rank and low precision factorization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–11.

[83] X. Wang, Y. Zheng, Z. Wan, and M. Zhang, "SVD-LLM: Truncation-aware singular value decomposition for large language model compression," 2024, *arXiv:2403.07378*.

[84] P. Sharma, J. T. Ash, and D. Misra, "The truth is in there: Improving reasoning in language models with layer-selective rank reduction," 2023, *arXiv:2312.13558*.

[85] B. Chen, H. Chen, J. He, F. Sun, and S. Jui, "Ternary singular value decomposition as a better parameterized form in linear mapping," 2023, *arXiv:2308.07641*.

[86] M. Xu, Y. L. Xu, and D. P. Mandic, "TensorGPT: Efficient compression of large language models based on tensor-train decomposition," 2023, *arXiv:2307.00526*.

[87] M. Zaheer et al., "Big bird: Transformers for longer sequences," in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 17283–17297.

[88] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," 2019, *arXiv:1904.10509*.

[89] A. Roy, M. Saffar, A. Vaswani, and D. Grangier, "Efficient content-based sparse attention with routing transformers," *Trans. Assoc. for Comput. Linguistics*, vol. 9, pp. 53–68, Feb. 2021.

[90] M. Lee, K. Han, and M. C. Shin, "LittleBird: Efficient faster & longer transformer for question answering," 2022, *arXiv:2210.11870*.

[91] T. Dao et al., "Monarch: Expressive structured matrices for efficient and accurate training," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2022, pp. 4690–4721.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

[92] Y. Tay, D. Bahri, L. Yang, D. Metzler, and D.-C. Juan, "Sparse Sinkhorn attention," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 9438–9447.

[93] X. Li, Y. Meng, M. Zhou, Q. Han, F. Wu, and J. Li, "SAC: Accelerating and structuring self-attention via sparse adaptive connection," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 16997–17008.

[94] L. Huang, S. Cao, N. Parulian, H. Ji, and L. Wang, "Efficient attentions for long document summarization," 2021, *arXiv:2104.02112*.

[95] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5156–5165.

[96] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-attention with linear complexity," 2020, *arXiv:2006.04768*.

[97] Z. Shen, M. Zhang, H. Zhao, S. Yi, and H. Li, "Efficient attention: Attention with linear complexities," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 3531–3539.

[98] K. Choromanski et al., "Rethinking attention with performers," 2020, *arXiv:2009.14794*.

[99] X. Ma et al., "Luna: Linear unified nested attention," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 2441–2453.

[100] I. Han, R. Jayaram, A. Karbasi, V. Mirrokni, D. P. Woodruff, and A. Zandieh, "HyperAttention: Long-context attention in near-linear time," 2023, *arXiv:2310.05869*.

[101] A. Zandieh, I. Han, M. Daliri, and A. Karbasi, "KDEformer: Accelerating transformers via kernel density estimation," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 40605–40623.

[102] P. Kacham, V. Mirrokni, and P. Zhong, "PolySketchFormer: Fast transformers via sketching polynomial kernels," 2023, *arXiv:2310.01655*.

[103] C.-F. R. Chen, Q. Fan, and R. Panda, "CrossViT: Cross-attention multi-scale vision transformer for image classification," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 357–366.

[104] Y. Xiong et al., "Nyströmformer: A Nyström-based algorithm for approximating self-attention," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 16, pp. 14138–14148.

[105] Z. Xia, X. Pan, S. Song, L. E. Li, and G. Huang, "Vision transformer with deformable attention," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 4794–4803.

[106] X. Ma et al., "Mega: Moving average equipped gated attention," 2022, *arXiv:2209.10655*.

[107] W. Hua et al., "Transformer quality in linear time," in *Proc. 39th Int. Conf. Mach. Learn.*, vol. 162, 2022, pp. 9099–9117.

[108] T. J. Ham et al., "A3: Accelerating attention mechanisms in neural networks with approximation," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, May 2020, pp. 328–341.

[109] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 16344–16359.

[110] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," 2023, *arXiv:2307.08691*.

[111] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th Symp. Operating Syst. Princ.*, Oct. 2023, pp. 611–626.

[112] X. Wang, Y. Xiong, Y. Wei, M. Wang, and L. Li, "LightSeq: A high performance inference library for transformers," 2020, *arXiv:2010.13887*.

[113] T. J. Ham et al., "ELSA: Hardware–software co-design for efficient, lightweight self-attention mechanism in neural networks," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2021, pp. 692–705.

[114] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," 2021, *arXiv:2111.00396*.

[115] H. Mehta, A. Gupta, A. Cutkosky, and B. Neyshabur, "Long range language modeling via gated state spaces," 2022, *arXiv:2206.13947*.

[116] A. Gupta, "Diagonal state spaces are as effective as structured state spaces," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 22982–22994.

[117] L. Ren, Y. Liu, S. Wang, Y. Xu, C. Zhu, and C. Zhai, "Sparse modular activation for efficient sequence modeling," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–12.

[118] M. Fathi, J. Pilault, O. Firat, C. Pal, P.-L. Bacon, and R. Goroshin, "Block-state transformers," 2023, *arXiv:2306.09539*.

[119] D. Y. Fu, T. Dao, K. K. Saab, A. W. Thomas, A. Rudra, and C. Ré, "Hungry hungry hippos: Towards language modeling with state space models," 2022, *arXiv:2212.14052*.

[120] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.

[121] S. Zhai et al., "An attention free transformer," 2021, *arXiv:2105.14103*.

[122] A. Bulatov, Y. Kuratov, and M. Burtsev, "Recurrent memory transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 11079–11091.

[123] L. Yu, D. Simig, C. Flaherty, A. Aghajanyan, L. Zettlemoyer, and M. Lewis, "MEGABYTE: Predicting million-byte sequences with multiscale transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–7.

[124] M. Poli et al., "Hyena hierarchy: Towards larger convolutional language models," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 28043–28078.

[125] A. Orvieto et al., "Resurrecting recurrent neural networks for long sequences," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 26670–26698.

[126] B. Peng et al., "RWKV: Reinventing RNNs for the transformer era," 2023, *arXiv:2305.13048*.

[127] C. Xiao et al., "Configurable foundation models: Building LLMs from a modular perspective," 2024, *arXiv:2409.02877*.

[128] D. Lepikhin et al., "GShard: Scaling giant models with conditional computation and automatic sharding," 2020, *arXiv:2006.16668*.

[129] C. Riquelme et al., "Scaling vision with sparse mixture of experts," in *Proc. NIPS*, 2021, pp. 8583–8595.

[130] B. Mustafa, C. Riquelme, J. Puigcerver, R. Jenatton, and N. Houlsby, "Multimodal contrastive learning with LIMoE: The language-image mixture of experts," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 9564–9576.

[131] N. Du et al., "GLaM: Efficient scaling of language models with mixture-of-experts," in *Proc. Int. Conf. Mach. Learn.*, vol. 162, 2022, pp. 5547–5569.

[132] A. Bakhtiarnia, Q. Zhang, and A. Iosifidis, "Multi-exit vision transformer for dynamic inference," 2021, *arXiv:2106.15183*.

[133] T. Schuster et al., "Confident adaptive language modeling," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 17456–17472.

[134] L. Del Corro, A. Del Giorno, S. Agarwal, B. Yu, A. Awadallah, and S. Mukherjee, "SkipDecode: Autoregressive skip decoding with batching and caching for efficient LLM inference," 2023, *arXiv:2307.02628*.

[135] Z. Zeng, Y. Hong, H. Dai, H. Zhuang, and C. Chen, "ConsistentEE: A consistent and hardness-guided early exiting method for accelerating language models inference," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 17, pp. 19506–19514.

[136] S. Bae, J. Ko, H. Song, and S.-Y. Yun, "Fast and robust early-exiting framework for autoregressive language models with synchronized parallel decoding," 2023, *arXiv:2310.05424*.

[137] J. Yuan et al., "Rethinking mobile AI ecosystem in the LLM era," 2023, *arXiv:2308.14363*.

[138] G. Xiao et al., "DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads," 2024, *arXiv:2410.10819*.

[139] A. Mohtashami and M. Jaggi, "Random-access infinite context length for transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–11.

[140] Z. Liu et al., "Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–8.

[141] S. Anagnostidis, D. Pavllo, L. Biggio, L. Noci, A. Lucchi, and T. Hoffmann, "Dynamic context pruning for efficient and interpretable autoregressive transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–9.

[142] Z. Zhang et al., "H_2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Dec. 2023, pp. 1–17.

[143] H. Wang, Z. Zhang, and S. Han, "SpAtten: Efficient sparse attention architecture with cascade token and head pruning," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Feb. 2021, pp. 97–110.

[144] S. Kim et al., "Learned token pruning for transformers," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 784–794.

[145] Y. Guan, Z. Li, J. Leng, Z. Lin, and M. Guo, "Transkimmer: Transformer learns to layer-wise skim," 2022, *arXiv:2205.07324*.

[146] W. Tan, "Infor-coef: Information bottleneck-based dynamic token downsampling for compact and efficient language model," 2023, *arXiv:2305.12458*.

[147] Q. Cao, B. Paranjape, and H. Hajishirzi, "PuMer: Pruning and merging tokens for efficient vision language models," 2023, *arXiv:2305.17530*.

[148] Z. Wang et al., "SmartTrim: Adaptive tokens and attention pruning for efficient vision-language models," 2023, *arXiv:2305.15033*.

[149] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, and C. J. Hsieh, "DynamicViT: Efficient vision transformers with dynamic token sparsification," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2021, pp. 13937–13949.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.

[150] L. Meng et al., "AdaViT: Adaptive vision transformers for efficient image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 12309–12318.
[151] H. Yin, A. Vahdat, J. M. Alvarez, A. Mallya, J. Kautz, and P. Molchanov, "A-ViT: Adaptive tokens for efficient vision transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 10809–10818.
[152] Z. Kong et al., "SPViT: Enabling faster vision transformers via latency-aware soft token pruning," in *Proc. 17th Eur. Conf. Comput. Vis. ECCV*, 2022, pp. 620–640.
[153] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 19274–19286.
[154] Y. Zhou et al., "DistillSpec: Improving speculative decoding via knowledge distillation," 2023, *arXiv:2310.08461*.
[155] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," 2023, *arXiv:2302.01318*.
[156] X. Liu et al., "Online speculative decoding," 2023, *arXiv:2310.07177*.
[157] W. Qi et al., "ProphetNet: Predicting future N-gram for sequence-to-sequence pre-training," 2020, *arXiv:2001.04063*.
[158] T. Cai et al., "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," 2024, *arXiv:2401.10774*.
[159] Y. Fu, P. Bailis, I. Stoica, and H. Zhang, "Break the sequential dependency of LLM inference using lookahead decoding," 2024, *arXiv:2402.02057*.
[160] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," 2024, *arXiv:2401.15077*.
[161] N. Yang et al., "Inference with reference: Lossless acceleration of large language models," 2023, *arXiv:2304.04487*.
[162] Z. He, Z. Zhong, T. Cai, J. D. Lee, and D. He, "REST: Retrieval-based speculative decoding," 2023, *arXiv:2311.08252*.
[163] T. Lan, D. Cai, Y. Wang, H. Huang, and X.-L. Mao, "Copy is all you need," 2023, *arXiv:2307.06962*.
[164] (2023). *DeepSpeed*. [Online]. Available: https://github.com/microsoft/DeepSpeed/tree/master
[165] (2023). *LightLLM*. [Online]. Available: https://github.com/ModelTC/lightllm
[166] (2023). *Llama.CPP*. [Online]. Available: https://github.com/ggerganov/llama.cpp
[167] (2023). *MNN-LLM*. [Online]. Available: https://github.com/wangzhaode/mnn-llm
[168] (2023). *MLLM*. [Online]. Available: https://github.com/UbiquitousLearning/mllm
[169] A. Zeng et al., "ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools," 2024, *arXiv:2406.12793*.
[170] J. Bai et al., "Qwen technical report," 2023, *arXiv:2309.16609*.
[171] A. Gopalkrishnan, R. Greer, and M. Trivedi, "Multi-frame, lightweight & efficient vision-language models for question answering in autonomous driving," 2024, *arXiv:2403.19838*.
[172] J. Xu, C. Lan, W. Xie, X. Chen, and Y. Lu, "Slot-VLM: SlowFast slots for video-language modeling," 2024, *arXiv:2402.13088*.
[173] W. Hu, Y. Xu, Y. Li, W. Li, Z. Chen, and Z. Tu, "BLIVA: A simple multimodal LLM for better handling of text-rich visual questions," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 3, pp. 2256–2264.
[174] H. Liu, C. Tian, X. Wei, Q. Li, and L. Li, "Runtime adaptive pruning for LLM inference," 2025, *arXiv:2505.17138*.
[175] N. Zhang et al., "Pruning as a domain-specific LLM extractor," 2024, *arXiv:2405.06275*.
[176] J. Lim, I. Sa, B. MacDonald, and H. S. Ahn, "A sign language recognition system with pepper, lightweight-transformer, and LLM," 2023, *arXiv:2309.16898*.
[177] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.
[178] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
[179] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 1–12.
[180] N. Shazeer, "GLU variants improve transformer," 2020, *arXiv:2002.05202*.
[181] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, Feb. 2024, Art. no. 127063.
[182] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 240, pp. 1–113, 2022.
[183] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.
[184] Y. Li, L. Niu, X. Zhang, K. Liu, J. Zhu, and Z. Kang, "E-sparse: Boosting the large language model inference through entropy-based N:M sparsity," 2023, *arXiv:2310.15929*.
[185] Y. Zhang, H. Bai, H. Lin, J. Zhao, L. Hou, and C. V. Cannistraci, "An efficient plug-and-play post-training pruning strategy in large language models," Dept. Comput. Sci., Huawei Noah's Ark Lab, Inst. Automat., Chin. Acad. Sci., Beijing, China, Tech. Rep., 2023.
[186] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
[187] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for thin deep nets," 2014, *arXiv:1412.6550*.
[188] N. Passalis, M. Tzelepi, and A. Tefas, "Probabilistic knowledge transfer for lightweight deep representation learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 5, pp. 2030–2039, May 2021.
[189] J. Kim, S. Park, and N. Kwak, "Paraphrasing complex network: Network compression via factor transfer," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 1–15.
[190] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," 2020, *arXiv:2004.05150*.
[191] N. Kitaev, Ł. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," 2020, *arXiv:2001.04451*.
[192] Y. Sun et al., "Retentive network: A successor to transformer for large language models," 2023, *arXiv:2307.08621*.
[193] M. Artetxe et al., "Efficient large scale language modeling with mixtures of experts," 2021, *arXiv:2112.10684*.
[194] Y. Liu et al., "CacheGen: KV cache compression and streaming for fast large language model serving," in *Proc. ACM SIGCOMM Conf.*, Aug. 2024, pp. 38–56.
[195] L. Ma et al., "Defying multi-model forgetting in one-shot neural architecture search using orthogonal gradient learning," *IEEE Trans. Comput.*, vol. 74, no. 5, pp. 1678–1689, May 2025.
[196] G. Park et al., "LUT-GEMM: Quantized matrix multiplication based on LUTs for efficient inference in large-scale generative language models," in *Proc. 12th Int. Conf. Learn. Represent.*, 2022, pp. 1–12.
[197] S. Kim et al., "SqueezeLLM: Dense-and-sparse quantization," 2023, *arXiv:2306.07629*.
[198] Z. Yuan et al., "RPTQ: Reorder-based post-training quantization for large language models," 2023, *arXiv:2304.01089*.
[199] C. Guo et al., "OliVe: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proc. 50th Annu. Int. Symp. Comput. Archit.*, Jun. 2023, pp. 1–15.
[200] S.-Y. Liu, Z. Liu, X. Huang, P. Dong, and K.-T. Cheng, "LLM-FP4: 4-bit floating-point quantized transformers," 2023, *arXiv:2310.16836*.
[201] W. Shao et al., "OmniQuant: Omnidirectionally calibrated quantization for large language models," 2023, *arXiv:2308.13137*.
[202] Y. Bondarenko, M. Nagel, and T. Blankevoort, "Quantizable transformers: Removing outliers by helping attention heads do nothing," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–8.
[203] Y. Yue, Z. Yuan, H. Duanmu, S. Zhou, J. Wu, and L. Nie, "WKVQuant: Quantizing weight and key/value cache for large language models gains more," 2024, *arXiv:2402.12065*.
[204] Y. Zhang et al., "Dynamic sparse no training: Training-free fine-tuning for sparse LLMs," 2023, *arXiv:2310.08915*.
[205] H. Shao, B. Liu, B. Xiao, K. Zeng, G. Wan, and Y. Qian, "One-shot sensitivity-aware mixed sparsity pruning for large language models," 2023, *arXiv:2310.09499*.
[206] Y. Ji, Y. Cao, and J. Liu, "Pruning large language models via accuracy predictor," 2023, *arXiv:2309.09507*.
[207] S. Ashkboos, M. L. Croci, M. G. Nascimento, T. Hoefler, and J. Hensman, "SliceGPT: Compress large language models by deleting rows and columns," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024, pp. 1–16.
[208] Z. Yuan et al., "ASVD: Activation-aware singular value decomposition for compressing large language models," 2023, *arXiv:2312.05821*.
[209] A. Liu et al., "DeepSeek-V3 technical report," 2024, *arXiv:2412.19437*.
[210] L. Ma, S. Cheng, and Y. Shi, "Enhancing learning efficiency of brain storm optimization via orthogonal learning design," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 51, no. 11, pp. 6723–6742, Nov. 2021.
[211] L. Ma et al., "TBCIM: Two-level blockchain-aided edge resource allocation mechanism for federated learning service market," *IEEE Trans. Netw.*, early access, Jul. 29, 2025, doi: 10.1109/TON.2025.3589017.
[212] D. Guo et al., "DeepSeek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.
[213] *Grok-4*. Accessed: Jul. 2025. [Online]. Available: https://x.ai/news/grok-4
[214] A. Hurst et al., "GPT-4o system card," 2024, *arXiv:2410.21276*.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.



Jian Cheng received the B.Sc. degree in automation, the M.Sc. degree in control theory and control engineering, and the Ph.D. degree in communication and information system from China University of Mining and Technology, Xuzhou, China, in 1997, 2003, and 2008, respectively.

He has been a Post-Doctoral Fellow with Tsinghua University, Beijing, China, and the University of Birmingham, Birmingham, U.K., from 2009 to 2013. He is currently a Professor and the Chief Scientist with the Research Institute of Mine Artificial Intelligence, Chinese Institute of Coal Science, Beijing. His current research interests include machine learning and pattern recognition, data mining and big data, as well as imbalance learning and image processing, and their applications in industrial fields.



Haidong Kang (Student Member, IEEE) received the M.S. and Ph.D. degrees in software engineering from Northeastern University, Shenyang, China, in 2020 and 2025, respectively.

His work appeared in several top-tier ML and DL conferences, including International Joint Conference on Artificial Intelligence (IJCAI), International Conference on Computer Vision (ICCV), ACM Multimedia (ACM MM), IEEE TRANSACTIONS ON COMPUTERS, and IEEE TRANSACTIONS ON MOBILE COMPUTING. His current research interests include self-evolution AI, deep learning, AI for edge computing, and LLMs.

Dr. Kang served as the reviewer for World Wide Web Conference (WWW), Association for the Advancement of Artificial Intelligence (AAAI), IJCAI, International Conference on Machine Learning (ICML), ICCV, Knowledge Discovery and Data Mining (KDD), ACM MM, Neural Information Processing Systems (NeurIPS), International Conference on Learning Representations (ICLR), IEEE TRANSACTIONS ON IMAGE PROCESSING, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, and IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE.



Yuxin Shao received the M.S. degree in software engineering from Northeastern University, Shenyang, China, in 2025.

His current research interests include deep learning and computer vision.



Nan Li received the B.Sc. degree in software engineering from North University of China, Taiyuan, China, in 2020, and the Ph.D. degree in software engineering from Northeastern University, Shenyang, China, in 2025.

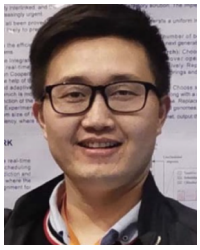
He is currently a Lecturer with the School of Computer and Information Science, Shanxi University, Taiyuan. His research interests include computational intelligence and machine learning.

Dr. Li serves as a reviewer for several international conferences, such as International Conference on Machine Learning (ICML) and International Joint Conference on Artificial Intelligence (IJCAI), and journals, including IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION and IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS.



Pengjun Chen is currently pursuing the bachelor's degree in software engineering with Northeastern University, Shenyang, China.

His current research interests include deep learning, computer vision, neural architecture search, and generative modeling.



Rui Wang received the bachelor's degree from the National University of Defense Technology, Changsha, China, in 2008, and the doctorate degree from the University of Sheffield, Sheffield, U.K., in 2013.

He is currently an Associate Professor with the National University of Defense Technology. His research interest includes evolutionary computation, multiobjective optimization, and the development of algorithms applicable in practice.

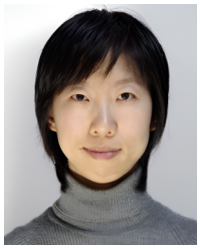
Dr. Wang was a recipient of the Operational Research Society Ph.D. Prize in 2016 and the National Science Fund for Outstanding Young Scholars in 2021. He is an Associate Editor for IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION and *Swarm and Evolutionary Computation*.



Saiqin Long (Member, IEEE) received the B.E. degree in software engineering from Hunan Normal University, Changsha, China, in 2009, and the Ph.D. degree in computer applications technology from South China University of Technology, Guangzhou, China, in 2014.

She was appointed as a Professor with the School of Computer Science, Xiangtan University, Xiangtan, China, in 2017. Her research interests include cloud computing, cloud storage, parallel and distributed systems, file systems, and computer system architecture.

Dr. Long is a member of Chinese Computer Federation (CCF).



Xiaochun Yang (Senior Member, IEEE) received the Ph.D. degree in computer software and theory from Northeastern University, Shenyang, China, in 2001.

She is currently a Professor with the School of Computer Science and Engineering, Northeastern University. Her research interests include big data management and knowledge engineering, data quality management, data privacy preserving, and recommender systems. More details about her research can be found at http://faculty.neu.edu.cn/yangxiaochun



Lianbo Ma (Senior Member, IEEE) received the B.S. degree in communication engineering and the M.S. degree in communication and information systems from Northeastern University, Shenyang, China, in 2004 and 2007, respectively, and the Ph.D. degree from the University of Chinese Academy of Sciences, Beijing, China, in 2015.

He is currently a Professor with Northeastern University. He has published over 90 journal articles, books, and refereed conference papers. His current research interests include computational intelligence and machine learning.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 09,2026 at 11:34:05 UTC from IEEE Xplore. Restrictions apply.